

Final Project Report

FocusFinder:
An adaptive approach to improving user productivity based on behavioural feedback patterns

TU857
BSc in Computer Science

Cian Anderson

C22793219

Supervisor:
Ciaran O Driscoll

School of Computer Science

Technological University, Dublin

Date:
17/04/2026

Abstract

This report presents FocusFinder, an adaptive productivity support system designed to help users maintain attention and focus during computer-based work. The system is particularly aimed at students and individuals in educational environments, with a specific emphasis on supporting users with Attention Deficit Hyperactivity Disorder (ADHD) and executive dysfunction.

FocusFinder operates by continuously monitoring desktop activity, including application usage, task switching, and idle behaviour, in order to classify user activity patterns. These behavioural insights are stored locally and used to generate personalised, research-informed reminders to improve focus and task engagement.

At the core of the system is a lightweight reinforcement learning model based on the Multi-Armed Bandit approach. This enables the system to dynamically adapt both the timing and tone of reminders based on user responses, allowing it to identify and prioritise the most effective feedback strategies over time.

Unlike traditional productivity tools that rely on static feedback or generic interventions, FocusFinder provides a behaviour-responsive system that evolves with the user. The design is grounded in research from cognitive science, behavioural psychology, and reinforcement learning, ensuring that feedback is supportive, non-intrusive, and aligned with the needs of individuals who have difficulty regulating attention.

The result is a privacy-focused, locally deployed system that transforms passive activity tracking into an adaptive, personalised productivity assistant to improve sustained attention and long-term behavioural outcomes.

Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:



Date:

17/04/2025

Acknowledgements

I would like to sincerely thank my project supervisors, Dr. Svetlana Hensman and Brian Gillespie, for their guidance and support throughout the development of this project.

I am especially grateful to Dr. Hensman for her supervision during the initial stages, and to Brian Gillespie for stepping in and providing direction during the later phases, including the interim prototype demonstration.

I would also like to extend my thanks to Ciarán O'Driscoll and Sean O'Leary for their continued support throughout the Final Year Project process. Their feedback, advice, and guidance were invaluable in shaping both the direction and quality of this work.

Finally, I would like to thank all lecturers and staff involved in the TU857 Computer Science Infrastructure programme for their teaching, support, and guidance over the past four years.

Table of Contents

Abstract **1**
Declaration 1
Signed: 2
Acknowledgements **3**
Table of Contents **4**
1. Introduction **1**
1.1 Project Background 1
1.2 Project Description 2
1.3 Project Aims and Objectives 4
1.4 Project Scope 5
1.4.1 In Scope 5
1.4.2 Out of Scope 5
1.5 Thesis Roadmap 7
2. Literature Review **8**
2.1 Introduction 8
2.2 Alternative Existing Solutions 8
2.3. Technologies Researched 15
2.3.1 Programming Languages and Frameworks 15
2.4. Other Relevant Research 21
2.4.1. Executive Dysfunction and Attention Regulation in ADHD 21
2.4.2. Motivation, Reward Processing, and Feedback Response in ADHD. 22
2.5 Existing Final Year Projects 24
2.5.1 Project 1 (MindGame): 24
2.5.2 Project 2 (Football Data Mining and Visualisation): 26
2.6. Conclusions 28
3. System Analysis **28**
3.1 System Overview 29
3.2 Requirements Gathering 31
3.2.1. Key Stakeholders 31
3.2.2. Requirements Collection Methods 31
3.3 Requirements Analysis 34
3.3.1 System Actors 34
3.3.2 Identified Use Cases 35
3.3.3 Analysis of System Behaviour 35
3.4 Logical Architecture of System 36
3.4.1 System structure 36
3.4.2 Data Flow 37
3.4.2 System Behaviour 39
3.5 Initial System Specification 41
3.5.1 Functional requirements 41
3.5.2 Non-functional requirements 42
3.6 Conclusions 43

4. System Design.....44
4.1 Introduction.....44
4.2. Software Methodology.....44
4.3 Project Planning.....45
4.3.1 Project Schedule.....46
4.4. Overview of System.....48
4.4.1 Technical Architecture.....48
4.4.2 Fulfilment of System Requirements.....51
4.5 Additional Design Representations.....53
4.5.1 User Interface Wireframes.....53
4.5.2 Sequence Diagram.....54
4.5 Conclusions.....55
5. System Development.....56
5.1. Introduction.....56
5.2 Component Development.....56
5.2.1 Monitoring and Activity Capture.....57
5.2.2 Activity Classification and Behaviour Interpretation.....59
5.2.3 Data Storage and Database Design.....62
5.2.4 Adaptive Reminder and Bandit System.....66
5.2.5 User Interface and Desktop Integration.....70
5.3 Deployment.....74
5.3 Conclusions.....75
6. Testing and Evaluation.....76
6.1. Introduction.....76
6.2. System Testing.....76
6.2.1 Testing Strategy.....76
6.2.2 Automated Backend Unit Testing.....77
6.2.5 Test Coverage Of Functional Requirements.....79
6.2.6 Test Coverage Of Non-Functional Requirements.....80
6.2.7 Summary of Testing.....81
6.3 System Evaluation.....82
6.3.1 Evaluation Against Project Objectives.....82
6.3.2 Key Strengths of the Completed Work.....89
6.4 Evaluation of Project Management.....90
6.4.1 Evaluate Project Plan.....90
6.4.2 Evaluate Project Execution.....90
6.5 Conclusion.....91
7. Conclusions and Future Work.....92
7.1. Introduction.....92
7.2. Conclusions.....92
7.3. Future Work.....95
References.....96
Appendix A: System Model and Analysis.....

A.1 Use Case Diagram.....
A.2 Use Case Narratives.....
Appendix B: Design.....
B.1 Full-size Technical Architecture Diagram.....
B.2 Sequence Diagram.....
Appendix C: Prompts Used with Gen AI.....
C.1 Accessibility and Writing Process Declaration.....
C.2 Use of AI (ChatGPT) in Report Development.....
C.2.2 Prompts Related to Clarity and Understanding.....
C.2.3 Prompts Related to Structure and Organisation.....
C.2.4 Prompts Related to Reassurance and Validation.....
C.3 Use of AI (ChatGPT/ CoPilot) in Project Design (Prompt Examples).....
C.3.1 Research Direction and Technology Selection.....
C.3.2 System Analysis (Brainstorming Only).....
C.3.3 Design Considerations (Conceptual Guidance Only).....
C.4 Use of AI (CoPilot) in Coding.....
C.4.1 Debugging and Error Handling.....
C.4.2 Implementation and Syntax Support.....
C.4.3 Refactoring and Readability.....
C.4.2 Refactoring and Readability.....

1. Introduction

1.1 Project Background

The ability to maintain sustained attention is fundamental to productivity and learning. However, in modern digital environments characterised by constant stimulation, immediate entertainment, and rapid information exchange, maintaining focus has become increasingly difficult.

Recent studies have documented a dramatic decline in attention spans over the past two decades. For example, Dr Gloria Mark, a leading researcher in digital behaviour at the University of California, found that the average time a person spends focused on a single screen dropped from around 2.5 minutes in the early 2000s to just 47 seconds by 2023, a decrease of nearly 70% (Mark et al., 2023). Similarly, a King's College London survey found that 49% of adults feel they now struggle more with concentration than they did a few years ago (King's College London, 2022). Together, these findings highlight a widespread shift in cognitive endurance, making it increasingly difficult for individuals, particularly younger generations, to stay engaged with demanding tasks for extended periods.

This challenge is further amplified for people with executive dysfunction and ADHD. Individuals with ADHD experience significantly greater difficulty with sustained attention, task initiation, and resistance to distraction due to impairments in executive functioning (Kallweit et al., 2021). According to ADHD Ireland, approximately 5% of adults in Ireland are estimated to have ADHD, which is one of several conditions associated with chronic difficulties in task initiation, organisation, and sustaining focus. Additionally, the Central Statistics Office reports that 3.3% of the Irish population experiences long-term issues with memory, learning, or concentration, suggesting that as much as 7–9% of the population may experience persistent focus-related challenges.

As someone diagnosed with ADHD, I have experienced these challenges firsthand. This perspective has informed the design and motivation of this project, particularly in identifying the limitations of existing productivity tools and the importance of adaptive, user-sensitive feedback.

Although numerous productivity tools exist, such as RescueTime, Habitica, Forest, Pomodoro-based timers, and various browser extensions, they follow the same pattern: they track behaviour, present statistics, and offer simple reminders. However, based on observed usage patterns and informal feedback from ADHD-focused communities, these tools rarely result in lasting behavioural change. Their core limitation is that they do not adapt to changing conditions. They treat every user the same, regardless of personal challenges, emotional state, or responsiveness to different types of reminders. They do not adjust tone, message style, frequency, urgency, or level of encouragement. They remain static systems in a context where users' needs are highly dynamic.

Research supports this gap. Studies show that tone and communication style can significantly influence motivation, particularly in individuals with ADHD, who are especially sensitive to how feedback is delivered (Staff et al., 2022). For example, negative or critical feedback lowered motivation and engagement in individuals with ADHD (Staff et al., 2022). Meanwhile, timely and adaptive feedback has been shown to improve engagement and behavioural outcomes in digital interventions (Kumar et al., 2023). Despite this, most existing productivity tools still rely on fixed, one-size-fits-all approaches and do not incorporate adaptive or personalised feedback mechanisms.

Given this gap in behaviour-responsive design, there is a clear need for productivity support systems that can adapt dynamically to individual users. Such systems must go beyond passive tracking and static reminders, instead incorporating mechanisms that learn from user behaviour and adjust feedback accordingly. This requires an approach that balances responsiveness, personalisation, and computational efficiency, particularly in real-time desktop environments.

1.2 Project Description

FocusFinder is an adaptive productivity support system designed to run locally on a user's personal computer. The system continuously monitors user activity, including active applications, idle time, and task-switching behaviour, storing this data securely on the device. By operating entirely locally, it does not transmit data externally, ensuring user privacy is maintained at all times.

At the core of the system is a reinforcement learning algorithm based on the Multi-Armed Bandit model. This model selects and evaluates prompts and reminders from a predefined set, derived from behavioural research and informed by my lived experience with ADHD, including years spent developing and refining personal strategies to manage focus in educational settings and improve user focus and attention. Each prompt is treated as an "arm" within the model, and its effectiveness is measured by observing subsequent user behaviour over a defined evaluation period, with behaviour classified as productive or non-productive. This feedback is used to update the model's decision-making. Over time, the system prioritises strategies that consistently improve focus while continuing to explore alternative approaches to avoid stagnation.

Alongside the adaptive feedback system, the application provides a user-facing dashboard designed for clarity and usability. This dashboard presents key information, including activity statistics, productivity trends, and task-tracking features. The purpose of this interface is not only to visualise user behaviour but also to reinforce behavioural awareness and support self-regulation.

The system is designed with a focus on usability for individuals with ADHD and executive dysfunction, prioritising the needs of those who experience the

greatest difficulty in maintaining focus. This approach is based on the principle that a system effective under the most challenging conditions will remain effective for a broader user base. As a result, while FocusFinder is broadly applicable, its design accommodates high-variability attention patterns and reduced task persistence. Unlike traditional productivity tools that rely on static tracking or fixed reminders, FocusFinder is an adaptive system that learns from user interactions, delivering personalised feedback that evolves over time to support sustained productivity.

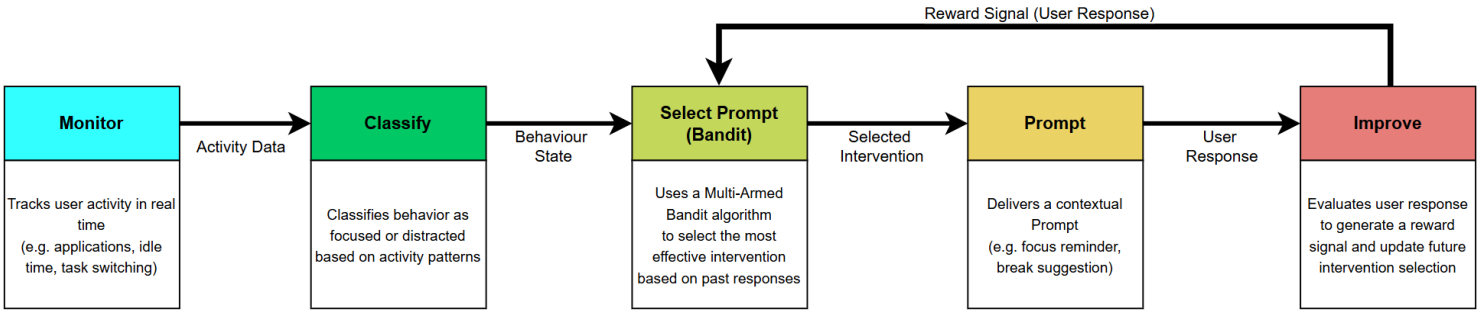


Figure 1: Adaptive feedback loop
Diagram of FocusFinder's adaptive feedback loop showing how user activity is monitored and classified; a multi-armed bandit selects personalised interventions; and user responses generate a reward signal that continuously improves decision-making.

1.3 Project Aims and Objectives

Aims:
This project aims to design and develop an adaptive productivity support system that monitors user behaviour and delivers personalised interventions. The system uses reinforcement learning, specifically a Multi-Armed Bandit approach, to optimise reminders over time, with a focus on supporting attention and task engagement.

Objectives:

1. **Conduct a research-informed design process:**
   Conduct a targeted literature review of ADHD, attention, feedback mechanisms, and reinforcement learning to inform system design decisions.
2. **Implement real-time user activity monitoring:**
   Develop a system to monitor user activity, including active applications, idle time, and task-switching behaviour.
3. **Store user data locally and securely:**
   Design and implement a local data storage solution to ensure all user activity data remains on-device and is not transmitted externally.
4. **Classify user behaviour:**
   Develop a mechanism to classify user activity into behavioural states (e.g. focused or distracted) to support system decision-making.
5. **Implement adaptive intervention selection:**
   Implement a Multi-Armed Bandit algorithm to select personalised interventions based on past user responses.
6. **Deliver contextual interventions:**
   Design and implement a system to deliver contextual prompts or interventions to improve user focus.
7. **Implement a feedback-driven learning loop:**
   Develop a feedback mechanism that evaluates user responses and generates a reward signal to continuously improve intervention selection.
8. **Evaluate system behaviour and effectiveness:**
   Evaluate the system using logged behavioural data and testing to assess its ability to adapt and influence user focus.

All objectives were achieved through the design, implementation, and evaluation of the FocusFinder system.

1.4 Project Scope

1.4.1 In Scope

- **User Activity Monitoring**
  Tracking desktop user activity, including application usage, idle time, and task-switching behaviour.

- **Local Data Storage**
  Storing all user activity data locally using a lightweight data storage solution.

- **Behaviour Classification**
  Categorising periods of activity as productive or non-productive based on observed usage patterns and user-defined rules.

- **Data Visualisation**
  Displaying tracked data through a simple interface, presenting basic metrics such as activity patterns and time usage.

- **Adaptive Intervention Selection:**
  Implementing a Multi-Armed Bandit model to select personalised interventions based on observed user behaviour and past responses.

- **Intervention Delivery:**
  Delivering adaptive prompts that respond to changes in user behaviour over time.

- **Feedback-Based Learning:**
  Implementing a feedback mechanism that evaluates user responses and generates a reward signal to update intervention selection over time.

- **System Evaluation:**
  Conducting a limited evaluation using system data and testing to assess system behaviour and usability.

- **Privacy-Focused Design:**
  Ensuring all user data remains local to the device, with no external transmission or cloud integration.

1.4.2 Out of Scope

- **Mobile Version:**
  Developing a mobile version of the system is excluded due to platform complexity, primarily due to platform-specific data access challenges and time constraints.

- **Third-Party Integrations:**
  Integration with external tools such as email, calendars, or task managers is excluded to maintain focus on core system functionality. These

integrations would require an internet connection, increasing system complexity and introducing potential security risks to sensitive user data.

- **Broader Clinical Expansion:**
  Extending the system to support additional cognitive or clinical conditions beyond ADHD and executive dysfunction would require significantly more research and would diminish the effectiveness of a focused, purpose-driven system.
- **Multi-User Systems:**
  Supporting multiple users or shared tracking environments conflicts with the system's local-first design, which prioritises user privacy and data security.
- **Long-Term Evaluation:**
  Conducting long-term or controlled studies measuring productivity over extended periods (e.g. months or years) is constrained by time limitations and access to suitable testing resources.
- **Automated Prompt Generation:**
  Generating prompts dynamically using advanced models (e.g. large language models) is not included due to the complexity and resource requirements of implementing such systems.
- **Perfect Behaviour Classification:**
  Fully accurate classification of user behaviour as focused or distracted is not feasible, as context-dependent activities (e.g., educational vs distracting use of the same application) cannot be reliably distinguished within this project's constraints.

Justification

The scope reflects the practical constraints of a single-student project with limited time and resources. By focusing on a local, single-user system with an adaptive intervention model, the project prioritises depth over breadth. This approach enables the development of a functional, research-informed system while avoiding unnecessary complexity. The design emphasises adaptive feedback and user privacy to ensure alignment with the project's core objectives.

1.5 Thesis Roadmap

This section provides an overview of the dissertation's structure.

Chapter 1: Introduction

Introduces the problem of declining focus and productivity in digital environments, particularly for users with ADHD or executive dysfunction, and outlines the aims, objectives, scope, and overall structure of the project.

Chapter 2: Literature Review

Examines existing productivity tools and relevant academic research in areas such as ADHD, behavioural psychology, feedback design, and reinforcement learning, providing the research foundation that informs the system design.

Chapter 3: System Analysis

Defines system stakeholders and requirements, and develops an initial system model through use cases, data flow, and behavioural analysis, establishing how the system should operate.

Chapter 4: System Design

Presents the system architecture, development methodology, and design representations, including the technical structure, component interactions, and user interface design.

Chapter 5: System Development

Describes the system's implementation, detailing the development of core components such as activity monitoring, classification, data storage, adaptive feedback, and user interface integration.

Chapter 6: Testing and Evaluation

Evaluates the completed system through testing and analysis, validating functionality, assessing performance, and discussing strengths, limitations, and alignment with project objectives.

Chapter 7: Conclusions and Future Work

Summarises the outcomes of the project, reflects on its success and limitations, and outlines potential future improvements and areas for further development.

2. Literature Review

2.1 Introduction

This chapter presents the research foundations that were undertaken to determine the design of the system.
My Research focuses on studies in psychology, machine learning, and interaction design that directly inform the project's aims and objectives. Particular attention is given to the cognitive challenges associated with ADHD, behavioural reinforcement models, and distraction dynamics in digital environments.
By understanding how users, especially those with executive dysfunction, respond to different feedback mechanisms, we can design a system that adapts not only to user behaviour but also to their motivational and emotional states.
Several existing productivity-boosting software tools were examined to compare their technical strategies and approaches to user experience, and to identify areas for improvement.
To better understand the scope and complexity of a final-year project within the given time constraints, I examined previous final-year projects.

2.2 Alternative Existing Solutions

Several productivity and focus-support tools are currently available, each offering distinct methods for monitoring user activity, encouraging task engagement, or preventing distractions. By examining these existing solutions, their strengths and limitations can be analysed to identify gaps in current approaches and inform the design of more effective systems.

RescueTime

RescueTime is a widely used activity-tracking tool that monitors time spent on applications and websites. It automatically categorises activity, provides weekly productivity reports, and generates detailed reports.
The system operates passively in the background, requires minimal user input, and provides valuable insights into patterns such as peak productivity periods and the distribution of time across tasks.

One of RescueTime's key strengths is its ability to record and deliver comprehensive analytics without requiring manual tracking, allowing users to build a comprehensive passive database of their digital habits.

However, this strength is also RescueTime's major weakness, as, by functioning primarily as a passive analytics tool, it does not offer any adaptive or personalised interventions based on the data it's gathering. All feedback is presented as statistics, requiring users to interpret and determine the best

course of action themselves. This places a large cognitive burden on the user, which is a notable challenge for individuals with ADHD who may struggle with self-regulation and translating insights into action.

While RescueTime is effective in increasing awareness of user behaviour, it does not actively support behaviour modification. In contrast, the proposed system, FocusFinder, builds upon this foundation by incorporating adaptive feedback mechanisms that respond dynamically to user activity, aiming to support not only awareness but also sustained behavioural change.

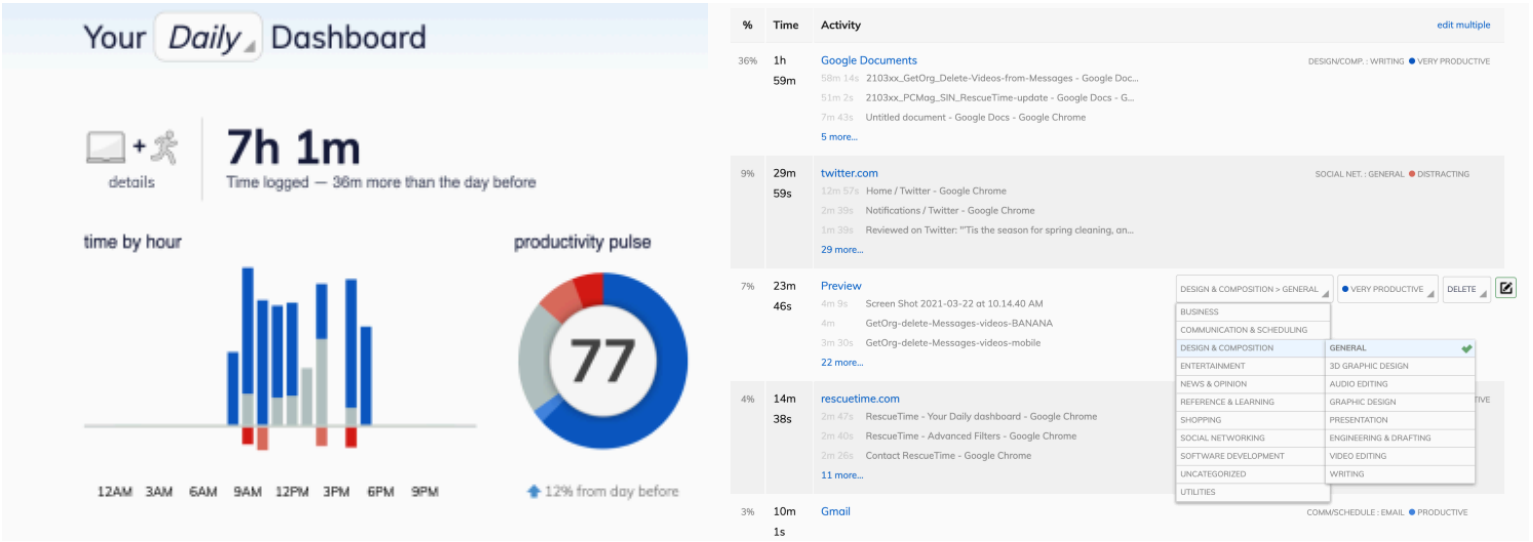


Figure 2: RescueTime Dashboard and Activity Categorisation

Screenshots of the RescueTime interface showing automated activity tracking and categorisation of user behaviour into productivity levels. The dashboard (Left) visualises the distribution of time and productivity metrics, while the activity view (Right) displays detailed logs of application and website usage.

Forest App

Forest is a productivity app that uses a gamified approach to encourage focus; Users plant a virtual tree that grows while they remain in the app, symbolising uninterrupted work. If the user exits the app before the session ends, the tree dies, introducing a simple consequence for distraction. Over time, users can build a visual “forest” representing their accumulated focus sessions.

One of Forest’s key strengths is its engaging, intuitive reward system. By transforming focus into a visual, interactive outcome, it provides immediate, tangible positive reinforcement that helps users stay on task. This approach is particularly effective for individuals with ADHD, as it leverages motivational strategies based on short-term rewards and visible progress.

However, Forest is primarily a mobile-based tool with a limited scope. It does not track broader user behaviour, particularly across desktop environments

where much productivity work occurs. Additionally, its reward mechanism is static, meaning the system does not adapt to individual user performance, habits, or changing levels of focus over time.

While Forest is highly effective at encouraging short-term focus through gamification, it lacks the ability to support long-term behavioural change. In contrast, FocusFinder extends this approach by incorporating behavioural tracking and adaptive feedback, enabling the system to respond dynamically to user activity and support sustained improvements in focus.



Figure 3: Forest Application Interface

Screenshots of the Forest mobile application illustrating its core features. The interface includes a timer-based focus session (Left), the visual growth of a virtual forest over time (Centre Left), basic usage statistics (Centre Right), and optional social-comparison features such as rankings (Right).

Habitica

Habitica is a productivity application that gamifies real-world tasks by representing them within a role-playing game environment. Users create and manage tasks such as habits, goals, and to-do lists, earning in-game rewards, including experience points, items, and character progression, upon completion. The system also incorporates social features, allowing users to collaborate in group challenges and quests, reinforcing motivation through shared progress and accountability.

A key strength of Habitica is its highly engaging reward system, which leverages established game mechanics such as progression, achievements, and social interaction to encourage consistent task completion. This approach can be

particularly effective for users who respond well to structured goals and external rewards, providing both a sense of accomplishment and motivation.

However, because Habitica is highly reliant on user-set goals, the entire system revolves around manual user input, requiring users to define, track, and update their own tasks without any automatic monitoring behaviour. This makes the system highly dependent on user discipline and consistency, which can be particularly challenging for individuals with ADHD. Furthermore, its reward and feedback systems are fixed and do not adapt based on user behaviour or effectiveness over time, limiting its ability to provide personalised support.

While Habitica is effective at motivating users through structured gamification and social reinforcement, it lacks behavioural awareness and adaptive feedback. The system inherently requires a baseline level of user discipline and the ability to recognise and respond to patterns in their own behaviour, which may be challenging for many users. In contrast, FocusFinder addresses these limitations by adapting its feedback based on observed user activity, reducing reliance on manual input and supporting users who may struggle to identify or respond to their own behavioural patterns.

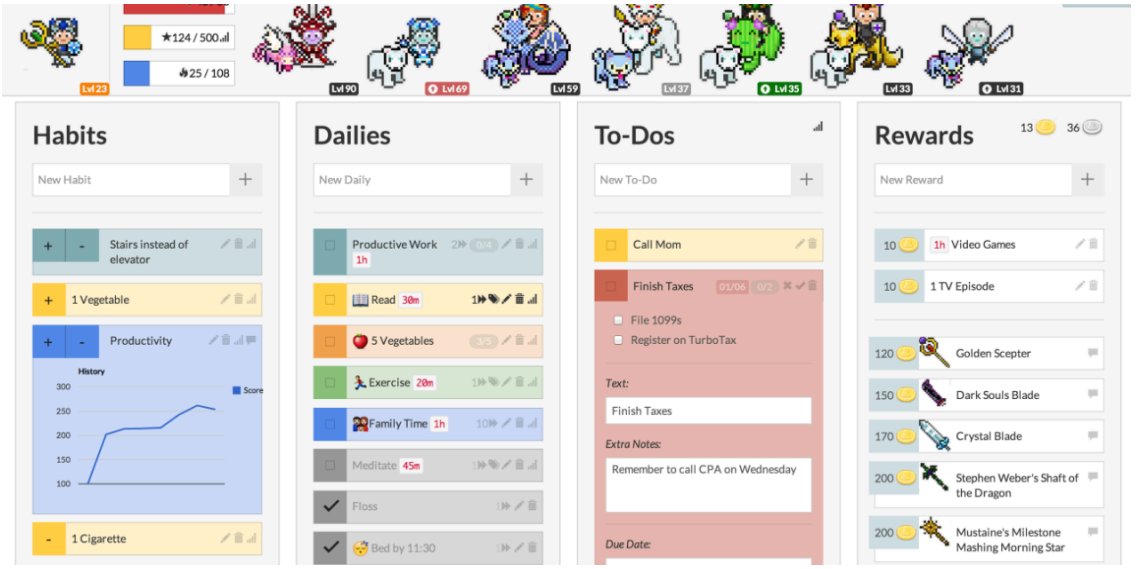


Figure 4: Habitica Task Management Interface

Screenshot of the Habitica application interface displaying task categories such as habits, daily goals, and to-do lists alongside a reward system based on in-game currency and items.

Pomofocus

Pomofocus is a web-based productivity tool that implements the Pomodoro Technique, a time management method that divides work into fixed intervals, typically 25 minutes of focused work followed by short breaks. Users can assign tasks, estimate the number of sessions required, and track their progress over time using built-in reporting features.

A key strength of the Pomodoro Technique is its effectiveness and simplicity in promoting structured work habits. Breaking tasks into measurable intervals, it helps users maintain focus and avoid burnout, particularly during extended work sessions.

However, Pomofocus operates entirely on a fixed, time-based structure and does not monitor or respond to actual user behaviour. If a user becomes distracted or switches tasks, the timer continues without adjustment, offering no awareness of whether meaningful work is being done.

While Pomofocus is effective at supporting structured time management, it addresses only one aspect of user focus and does not adapt to individual behaviour or attention patterns. In contrast, FocusFinder incorporates time-based structuring within a broader adaptive system, using behavioural monitoring to adjust feedback dynamically based on how the user is actually engaging with their work.

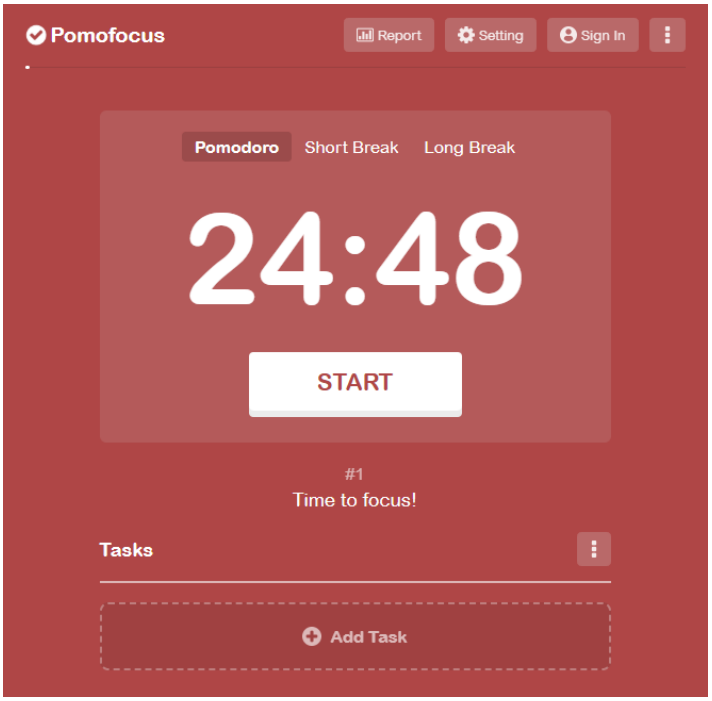


Figure 5: Pomofocus Timer-Based Interface

Screenshot of the Pomofocus interface displaying a timer-based focus session alongside task management features.

Freedom

Freedom is a productivity tool designed to reduce distractions by blocking access to selected websites or applications across multiple devices. Users can create custom blocklists, schedule focus sessions, and enable features such as Locked Mode, which prevents sessions from being interrupted once started. This approach enforces focus by removing access to common sources of distraction entirely.

A key strength of Freedom is its approach to improving focus. Whereas many other systems attempt to discourage users from using or getting distracted through various methods, Freedom eliminates distractions at their source by preventing access to time-wasting applications and websites. It reduces the reliance on user willpower and creates an environment more conducive to sustained focus.

However, this approach is highly restrictive and lacks nuance. By relying on the complete removal of distractions, the system does not support users in developing strategies to manage their own behaviour. Additionally, it does not provide feedback or insights into user activity, limiting its ability to support long-term improvements in focus. For some users, particularly those with ADHD, this rigid approach may be counterproductive and lead to frustration or avoidance.

While Freedom is effective at enforcing short-term focus through restriction, it does not support the development of long-term behavioural change. In contrast, FocusFinder adopts a more adaptive approach, supporting user development through behavioural monitoring and personalised feedback rather than relying solely on enforced limitations.

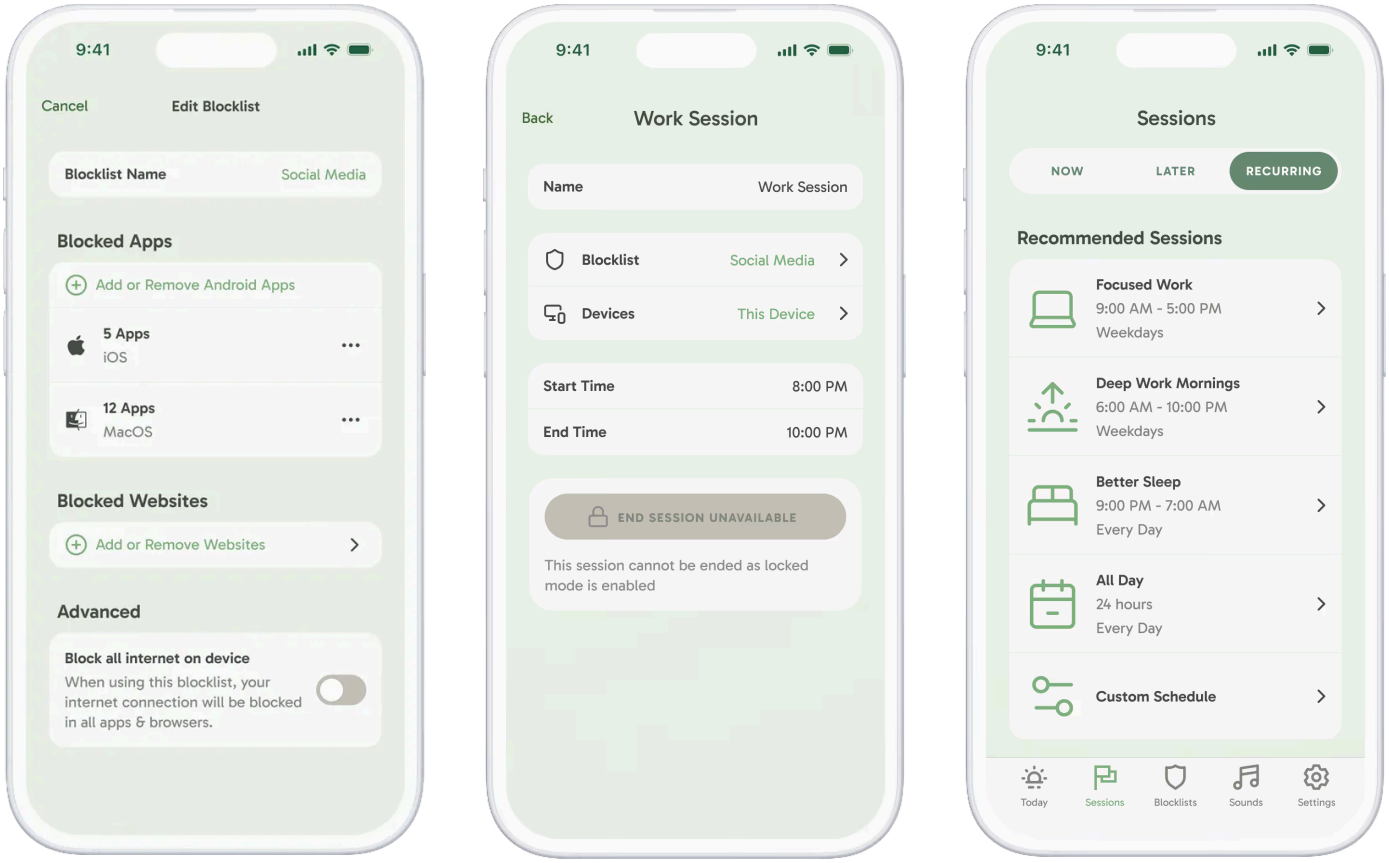


Figure 6: Freedom Blocklist and Session Scheduling Interface

Screenshots of the Freedom application interface illustrating its core functionality, including the creation of blocklists for applications and websites (left), configuration of locked focus sessions (centre), and scheduling of recurring or recommended focus periods (right).

2.3. Technologies Researched

The design of the system was shaped by research across multiple technical and behavioural domains, including reinforcement learning, system architecture, and user activity monitoring. The following subsections evaluate key technologies considered during development, outlining the rationale for the selected approaches and why alternative solutions were not adopted.

2.3.1 Programming Languages and Frameworks

Backend Logic and Processing Technologies:

Chosen: Python
Python was selected for its simplicity, rapid development capabilities, and a strong ecosystem for data processing and machine learning. It is particularly well-suited to implementing adaptive algorithms such as Multi-Armed Bandits while remaining lightweight enough to run continuously in the background. Its readability and extensive library support also make it ideal for developing and maintaining system logic.

Not Chosen: Java
Java was considered for its performance and robustness in large-scale systems. However, it introduces greater complexity, making it less suitable for rapid prototyping and lightweight background processes required by this system.

Not Chosen: C++
C++ offers high performance and low-level system control, which could be beneficial for resource-intensive applications. However, it significantly increases development complexity and reduces flexibility, making it less appropriate for a system prioritising adaptability and rapid iteration.

Frontend Interface Technologies

Chosen: React with TypeScript
React was selected for its component-based architecture, which enables efficient updates to constantly changing UI elements such as prompts and activity data. TypeScript improves reliability and maintainability through static typing, reducing runtime errors and improving scalability.

Not Chosen: Angular
Angular provides a comprehensive framework for frontend development, but it introduces additional complexity and overhead. For this project, its structure was considered unnecessarily heavy compared with the

flexibility offered by React.

Not Chosen: Vanilla JavaScript
While lightweight, vanilla JavaScript lacks the structure and scalability required to manage complex, state-driven interfaces. This would make maintaining and expanding the system more difficult over time.

Application Platform Technologies

Chosen: Electron
Electron was selected because it enables integration between frontend and backend components and provides access to operating system features, such as active window tracking. It allows the application to run locally and continuously, which is essential for real-time monitoring.

Not Chosen: Web-Based Application
A purely web-based solution was not suitable, as it cannot access system-level data such as active applications or idle time, which are critical to the system's functionality. In addition, internet connectivity introduces additional security risks.

2.3.2 Activity Monitoring and System Integration

Chosen: System-Level Activity Monitoring (Active Window Tracking and Idle Detection)
System-level activity monitoring was selected because it allows the application to track which programs are in use, how long they remain active, and when the user becomes idle. By integrating directly with the operating system, the system can continuously collect behavioural data without requiring manual user input, ensuring accurate and unobtrusive monitoring.
This is implemented using libraries such as pygetwindow for foreground window detection, pynput for keyboard and mouse activity tracking, and psutil and pywin32 for accessing system process information.

Not Chosen: Browser-Only Monitoring
Browser-based monitoring tools were considered, as they can track website usage and time spent online. However, this approach provides only partial visibility into user behaviour, as it cannot capture activity across desktop applications or system-level interactions. Given that productivity tasks often occur outside the browser, this method was insufficient for comprehensive behaviour analysis.

Not Chosen: Screen Capture / Image-Based Monitoring
Image-based monitoring approaches, such as periodically capturing screenshots and analysing them using image recognition techniques,

were also considered. While this method could provide a detailed view of user activity, it introduces significant drawbacks. These include high computational cost, increased system resource usage, and major privacy concerns due to the capture of potentially sensitive on-screen information. As a result, this approach was deemed unsuitable for a lightweight, privacy-focused system.

2.3.3 Local-First Architecture and Data Handling

Chosen: Local-First Architecture with Encrypted Local Storage
A fully local architecture was selected, with all user data stored on-device using SQLite accessed via SQLAlchemy ORM. This enables efficient storage of activity logs, user behaviour, and system interactions while maintaining low resource usage.
To enhance security, optional database encryption was implemented using SQLCipher, with encryption keys protected using the Windows Data Protection API (DPAPI) via CryptProtectData. This ensures that sensitive behavioural data remains secure at rest without requiring user-managed credentials.
All system communication is restricted to the local machine. The backend is implemented using Flask, ensuring no external network access is required.
Frontend preferences and lightweight user data are stored using browser-based localStorage, while generated reports are saved directly to the user's system using Electron's printToPDF() functionality. This design ensures that all data remains under user control and is never transmitted externally.

Not Chosen: Cloud-Based Architecture
Cloud-based systems were considered, as they allow for centralised data storage and more computationally intensive processing. However, this approach introduces significant privacy concerns, as sensitive behavioural data would need to be transmitted and stored on external servers. Additionally, reliance on network connectivity would reduce system reliability and responsiveness, particularly for real-time feedback systems.

Not Chosen: Hybrid (Local + Cloud) Storage
A hybrid approach, combining local storage with cloud synchronisation, was also considered. While this could provide backup and cross-device functionality, it introduces additional complexity and security risks. For a system focused on personal productivity and behavioural monitoring, maintaining complete local control over user data was prioritised over convenience features such as synchronisation.

2.3.4 Adaptive Decision Systems (Reinforcement learning)

To enable the system to adapt dynamically to user behaviour and feedback, a method for learning from user interactions is required. Static or rule-based approaches are insufficient, as they cannot adjust to individual user preferences or evolving behavioural patterns over time. As a result, a reinforcement learning approach was selected to enable continuous adaptation based on observed user responses.

Reinforcement learning (RL) is a branch of machine learning concerned with learning optimal behaviour through feedback and rewards. For adaptive feedback systems that must operate in real time, RL methods enable them to improve over time by adjusting their responses based on user interaction (Li et al., 2010).

Among RL approaches, Multi-Armed Bandit (MAB) algorithms were selected due to their simplicity, responsiveness, and suitability for environments with limited data and noisy feedback. In this model, each reminder strategy (e.g., gentle, neutral, direct) is treated as a separate "arm." The system selects one option, observes the user's response, and updates its future decisions accordingly. Over time, it learns which strategies yield the most effective outcomes for a specific user (Li et al., 2010).

This approach offers several advantages over more complex machine learning methods:

- **No training phase required**: Unlike deep learning models, MAB does not require a large dataset upfront. It starts adapting from the first use.
- **Low computational cost:** MAB algorithms are lightweight, allowing the assistant to run constantly in the background with minimal impact.
- **Interpretable behaviour:** MAB strategies are explainable and understood, enabling transparent justification of why the system chose a specific reminder style.
- **Suitable for personalised learning:** Because the algorithm adapts to each user individually, it enables highly targeted behaviour support without overfitting.

Previous research supports the effectiveness of this approach. Multi-Armed Bandit algorithms have been used in digital systems to optimise content and message selection in real time, improving user engagement and perceived relevance (Li et al., 2010). Similar approaches have also been applied in educational platforms to enhance personalised feedback and learning outcomes.

Not Chosen: Deep Reinforcement Learning / Neural Network-Based Approaches

More complex reinforcement learning approaches, such as Deep Reinforcement Learning (Deep RL), were considered as an alternative to Multi-Armed Bandit models. These methods use neural networks to model complex decision-making and can capture more nuanced behavioural patterns over time.

However, Deep RL was not selected due to several practical limitations. Firstly, such models require large amounts of training data to perform effectively, which is not available in a system that must begin adapting immediately from first use. Secondly, they introduce significant computational overhead, making them unsuitable for a lightweight application designed to run continuously in the background.

Additionally, Deep RL models are less interpretable than simpler approaches such as MAB. This lack of transparency makes it more difficult to justify or understand why specific feedback strategies are selected, which is undesirable in a system designed to provide clear and predictable behaviour.

Finally, the increased complexity of implementing and maintaining Deep RL models was not justified for the scope of this project, where a simpler, more efficient solution was sufficient to achieve effective adaptive behaviour.

2.3.5 Feedback Generation / Prompt System

Chosen: Rule-Based Prompt Templates

A rule-based prompt system was implemented using predefined message templates, categorised by function, including focus prompts, break reminders, preparation cues, and reward-based encouragement. These templates are defined within the system and provide consistent, controlled messaging across all interactions.
This approach ensures that all feedback remains appropriate in tone and avoids ambiguity or unpredictability. By using structured templates, the system can guarantee that prompts are concise, supportive, and aligned with the intended behavioural outcome. This is particularly important for maintaining clarity and avoiding overwhelming or negatively framed feedback.
Additionally, prompts are delivered through both visual notifications and auditory output via the Web Speech API, increasing accessibility and reducing reliance on constant visual attention.

Not Chosen: Dynamic Text Generation (LLM-Based Systems)

Dynamic text generation using large language models (LLMs) was considered as a method for producing highly flexible and context-aware feedback. However, this approach was not adopted due to several limitations. LLMs introduce variability in tone and content, which can lead to inconsistent or unpredictable messaging. Maintaining a controlled and supportive tone is critical for this system, making deterministic output preferable.

Not Chosen: Static, Non-Categorised Notifications

Basic notification systems with fixed, non-categorised messages were also considered. While simple to implement, they lack structure and flexibility, providing the same type of feedback regardless of context. This limits their effectiveness in supporting different stages of user activity, such as task initiation, sustained focus, or break reminders.

2.4. Other Relevant Research

This section examines additional research into the behavioural and psychological aspects of the system, while the previous section focused on the technological decisions that helped build it. It explores how users, particularly those with ADHD, interact with productivity systems. This research is primarily used to inform the design of the system's interaction model and user experience.

2.4.1. Executive Dysfunction and Attention Regulation in ADHD

ADHD is associated with executive dysfunction, characterised by difficulties with task initiation, sustained attention, and time management. Studies show that task initiation requires disproportionately more effort for individuals with ADHD, leading to procrastination and avoidance behaviours (Shaw et al., 2024). This manifests as impaired planning, difficulty with context switching, and frequent derailment once a task is initiated. Neuroimaging and behavioural data confirm that people with ADHD often have attenuated reward sensitivity at the start of tasks, contributing to inertia (Reiter et al., 2024).

Additionally, individuals with ADHD often exhibit impulsive task-switching, heightened distractibility, and poor responsiveness to traditional time-management tools. These behaviours are not purely attentional failures but are closely linked to deficits in self-regulation and executive control. Research indicates that individuals with ADHD struggle to maintain goal-directed behaviour over time, particularly in environments with competing stimuli or low immediate reward value (Hosseinnia et al., 2025). This contributes to frequent task abandonment and difficulty re-engaging with previously interrupted work.

These findings suggest that difficulties in ADHD are not limited to attention itself but also extend to the regulation of behaviour over time, in which individuals struggle to monitor, evaluate, and adjust their actions in response to changing task demands.

Digital environments with high information density further exacerbate context loss and overwhelm. Frequent notifications, multiple open applications, and rapid context switching increase cognitive load and make it harder to maintain a stable task representation. Experimental work shows that interruptions introduce a “resumption lag,” in which users require additional time and cognitive effort to return to their original task, thereby reducing overall efficiency (Wirzberger et al., 2024). For individuals with ADHD, this cost is likely amplified by already impaired attentional control. This aligns with cognitive load theory, where excessive information and frequent context switching can overwhelm working memory capacity, further reducing the ability to maintain task focus.

ADHD also co-occurs with emotional dysregulation, including heightened sensitivity to perceived criticism or negative feedback. Research shows that individuals with ADHD may interpret feedback more negatively and experience stronger emotional reactions, particularly when feedback is corrective or ambiguous (Schultz et al., 2022). Qualitative studies further demonstrate that criticism can be perceived indirectly, through comparison or tone, not just through explicit negative statements, reinforcing sensitivity to feedback framing (Beaton et al., 2022). This sustained exposure to perceived criticism has also been linked to reduced self-confidence and increased avoidance behaviours, further complicating engagement with structured productivity systems (Beaton et al., 2022).

Design implications:

- Supportive, well-timed nudges are more effective than punitive feedback.
- Immediate task-start assistance and visible progress markers are helpful.
- High-frequency task switching and idle time can be used to infer states of distraction.
- Minimising interruption cost and supporting task re-entry is critical for sustained focus.

2.4.2. Motivation, Reward Processing, and Feedback Response in ADHD

Individuals with ADHD tend to prefer immediate rewards to delayed gratification. In digital systems, feedback serves not only as information but also as a behavioural control mechanism that shapes user decision-making and engagement patterns (Steur and Seiter, 2021). Dopamine-mediated reinforcement is weaker for abstract, long-term outcomes (Baroni et al., 2018). This leads to reduced motivation for tasks that lack immediate feedback or visible progress, even when the long-term importance of the task is understood. As a result, traditional productivity systems that rely on delayed outcomes or retrospective summaries may fail to influence behaviour when it matters most.

Positive feedback, gamification, and encouragement are generally more effective than negative reinforcement, particularly for sustaining engagement and reducing avoidance behaviours. Research suggests that frequent, immediate, and clearly behaviour-linked reinforcement is more effective for individuals with ADHD than delayed or infrequent feedback (Kallweit et al., 2021). However, the relationship between feedback timing and effectiveness is nuanced. While immediate feedback can support behaviour correction, overly frequent or harsh corrective feedback may lead to emotional overload and disengagement (Baroni et al., 2018).

Classroom and behavioural studies reinforce that individuals with ADHD respond more positively to praise than to correction, and that excessive

corrective feedback can worsen behavioural outcomes (Schultz et al., 2022; Staff et al., 2023). In particular, individuals with ADHD are often exposed to disproportionately high levels of corrective feedback compared with positive reinforcement, which can reduce motivation and heighten sensitivity to feedback over time.

Timing is also critical to feedback effectiveness. Feedback must occur at a moment when the user can meaningfully act on it. Research on feedback encounters suggests that feedback is only effective when it aligns with the user's current task context and prompts meaningful action, as its impact depends on whether users engage with and apply the information provided (Jensen et al., 2025). Poorly timed feedback, even if accurate, may be ignored or fail to influence behaviour.

In digital interventions, static or generic reminders have limited effectiveness. For example, SMS reminder systems in ADHD interventions improved engagement only in certain cases, while overall adherence remained inconsistent, suggesting that generic reminders alone are insufficient to address the behavioural challenges associated with ADHD (Nordby et al., 2022). This suggests that reminder effectiveness depends not only on frequency but also on relevance, tone, and personalisation.

Adaptive systems offer a potential solution to this problem. Multi-Armed Bandit approaches enable systems to dynamically adjust feedback strategies based on user response, balancing exploration and exploitation to identify effective interventions over time (Li et al., 2010). This is particularly important in ADHD contexts, where behavioural responses to interventions can vary significantly between individuals and over time, making static feedback strategies insufficient. These approaches have been successfully applied in digital mental health systems to improve engagement and the perceived relevance of interventions (Kumar et al., 2024). Similarly, contextual bandit approaches have been used to tailor behavioural messaging based on user state, thereby improving motivation and engagement in real-time contexts (Song et al., 2025). However, repeated or poorly adapted interventions may lose effectiveness over time, a phenomenon referred to as intervention fatigue, further reinforcing the need for adaptive and context-aware systems (Song et al., 2025).

Design implications:

- Feedback tone must be adjustable to the user's preference.
- Soft, non-judgemental nudges help reduce emotional reactivity.
- Reminders should be framed as helpful prompts, not alerts or demands.
- Feedback should be timely, context-aware, and linked to observable behaviour.
- Adaptive systems can improve effectiveness by personalising feedback strategies over time.

2.5 Existing Final Year Projects

2.5.1 Project 1 (MindGame):

Title:
MindGame: A Machine Learning Framework for Electronic Games

Student:
Simon O'Neill, Dublin Institute of Technology, 2018

Description:
The project explores how a system can learn to play simple video games and accomplish in-game tasks. The machine learning system does this through reinforcement learning, using fixed observable actions and outcomes in the games. It does not rely on preexisting data but instead favours developing its own observations and using them to adapt its gameplay.

Relevance to this project:
This project is relevant to this project because it uses reinforcement learning to enable a system to adapt to observed outcomes. It demonstrates the implementation of a machine learning framework that operates without pre-existing data sets, relying instead on iterative learning through interaction. The project also highlights the challenges of designing a general-purpose reinforcement learning algorithm capable of handling a wide variety of inputs.

Complexity:
The project involved developing a general-purpose learning system from scratch in C#. The system takes data from previous runs of the game and rewards improvements defined by specific actions/goals. This involved, but was not limited to, data handling, mutation logic, performance evaluation, and rule generation. In addition, the system also needed to remain game-agnostic, allowing it to interact with a variety of games and mechanics through a standardised interface.

Technical architecture:
The system was built in C# using Microsoft Visual Studio, structured as a modular framework consisting of several key components:

> **Core Framework:** Handles the overall learning process, including running simulations, logging data, and coordinating between the game and learning algorithms.

> **Deciders:** Modules responsible for making in-game decisions based on learned heuristics or mutation-driven logic.

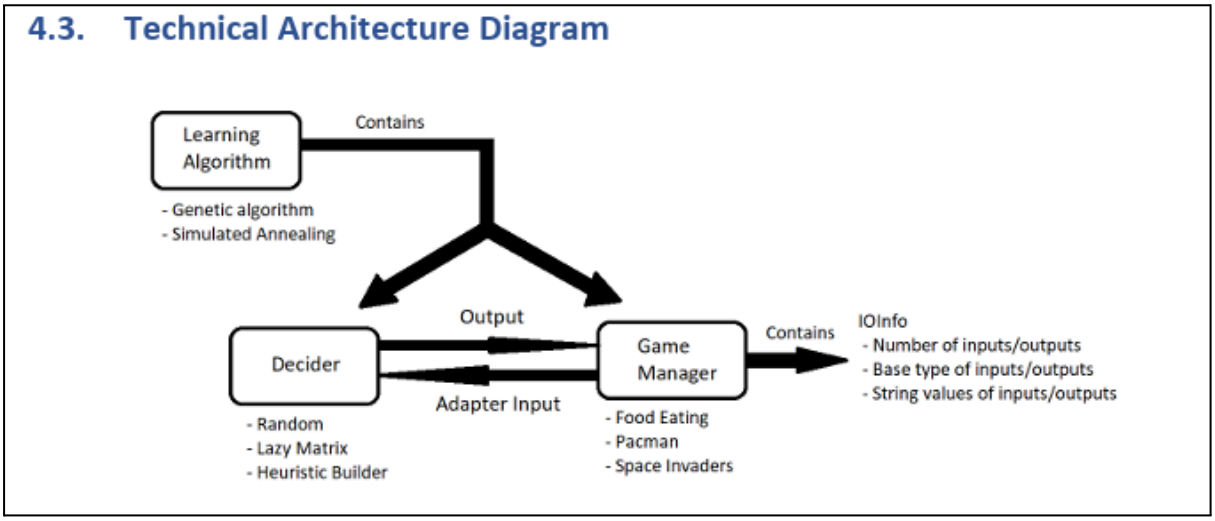
Learning Algorithms: Two custom algorithms, the Heuristic Building Decider (HBD) and Single Path Mutation Runner (SPMR), which analyse outcomes from previous runs and evolve strategies.

IO Adapters: Manage input and output between the learning system and the individual games, standardising communication through a common interface.

Game Interface (Tick System): Each game implements a tick() method, allowing the AI to progress game states in a controlled and measurable way (a tick refers to a frame/update in the game).

Logging and Analysis: Utilises Log4Net to record every iteration and output data in CSV format for later evaluation.

4.3. Technical Architecture Diagram



Key Strengths

- Demonstrates strong technical ability and a deep understanding of algorithm design with a custom machine learning framework in C#.
- Demonstrates significant design planning as the framework must translate diverse game rules, controls, and feedback into a single set of learning data. To use in the algorithm.
- Demonstrates the system's effectiveness by showing measurable learning improvements through iterative testing and evaluation.

Key Weaknesses

- The learning process was primarily mutation-driven, which limits its capacity for complex pattern recognition and long-term strategic adaptation. This could cause issues if it encounters problems late in the game that need to be resolved much earlier.

- The incomplete Space Invaders implementation restricted its ability to fully demonstrate its general-purpose potential across diverse game types.
- The system's reliance on repeated iterations for improvement resulted in high computational cost.
- The lack of a UX/UI makes it difficult to observe or interpret the AI's real-time decision-making behaviour. making debugging or analysing learning behaviour more challenging

2.5.2 Project 2 (Football Data Mining and Visualisation):

Title:
Football Data Mining, Result Prediction, and Visualisation

Student:
Yahia Ragab, Dublin Institute of Technology, 2018

Description :
In this project, Yahia developed a machine-learning-driven system to predict Premier League match outcomes. The project features a custom database built with PostgreSQL and hosted on AWS RDS, which stores match data, team statistics, and prediction results. A web and hybrid mobile app allows users to view predictions and match data through various charts. The Google Charts API is used to generate interactive visualisations from the collected statistics, and a Python-based machine learning classifier predicts match outcomes based on statistical and historical data.

Relevance to this project:
This project is relevant for integrating machine learning, data processing, and data visualisation into a single system. It demonstrates the use of structured data collection and analysis to generate meaningful visualisations, as well as the challenges of presenting complex data clearly and in a user-friendly manner.

Complexity:
This project focused on three major features: the machine learning classifier, data representation and collection, and the development of an app for both mobile and web-based platforms. Each presented its own challenges. The machine learning classifier, combined with the student's lack of prior experience, led to a lengthy process of selecting an appropriate classifier, requiring numerous experiments to determine the most effective one.

While obtaining the data and information necessary to run the machine learning algorithm was a challenge in itself, presenting this information coherently and integrating it with the Google Charts API for visualisation posed additional difficulties. Developing an application that functions effectively on both web and mobile platforms required considerably more work than focusing on a single platform, due to the differing specifications and requirements of each.

Technical architecture:
The system was developed using the Django web framework and structured as a layered architecture consisting of several key

Backend Framework: Django was used to handle the overall coordination between the machine learning model, the database, and the front-end interfaces. This managed data requests, predictions, and responses

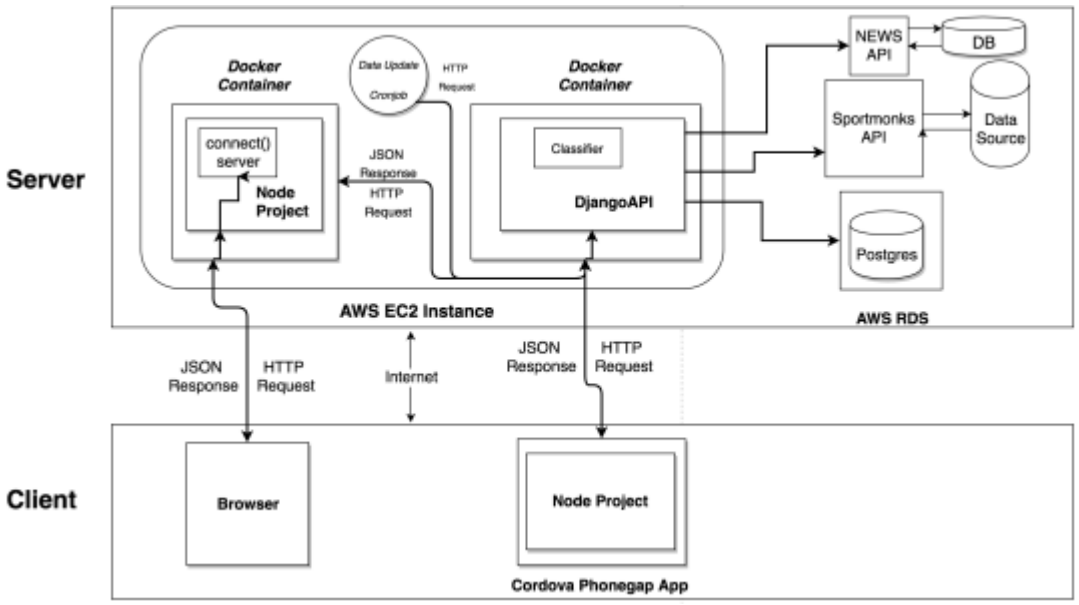
Machine Learning Module: Implemented using Python and then integrated into the back-end framework. The machine learning module processes statistical and historical data to generate predictions using a trained classifier.

Database: The database was developed using PostgreSQL. It stores match data, team statistics, and predictions from the machine learning module. It is hosted on AWS RDS to ensure its reliability and remote accessibility.

External Data APIs: Data was gathered using the SportsMonk API and the News API.

Visualisation: The Google Charts API was used to generate interactive visualisations in real-time for various purposes, including fixtures, team performance, and match predictions.

Web and Hybrid Mobile: A clean and user-friendly interface to allow the user to view prediction statistics and charts across multiple platforms.



Key Strengths

- The system is well-integrated across various modules, including machine learning, database management, and front-end development.
- A strong understanding of third-party integration is evident throughout the project, which uses multiple APIs (SportsMonk, Google Charts, News API).
- The entire project creation process is thoroughly documented.

Key Weaknesses

- The initial scope was far too big. A key example of this was the integration of VR. This led to delays throughout the project and a refocusing midway through development.
- The complexity of creating an app for both mobile and web versions significantly increased the development workload without a commensurate expansion of functionality.
- Their machine learning testing primarily focused on the model's usability and integration. However, they conducted limited testing of algorithmic bias or data validation.

2.6. Conclusions

The research in this chapter highlights several key behavioural, psychological, and technical insights that informed the development of the system.

Research into ADHD shows that users often struggle with task initiation, emotional sensitivity to feedback, and attention instability. These findings confirmed the need for a feedback system that avoids rigid, one-size-fits-all responses and instead adapts to the user’s state and preferences.

Research into reinforcement learning and Multi-Armed Bandit (MAB) algorithms demonstrated that this approach offers a lightweight, data-efficient solution for real-time personalisation.

Research into feedback tone and timing confirmed the importance of empathetic framing and interrupting only when helpful, which informed both the interface design and the delivery and wording of notifications.

Rather than treating productivity as a static measurement problem, the research supported a dynamic, feedback-driven model. This directly led to the system architecture being built around continuous learning, behavioural classification, and closed-loop interaction.

3. System Analysis

This chapter defines the analytical foundation upon which the Focus Finder system is built. It translates the research insights from the previous chapter into a structured understanding of how the system should operate, who it is designed for, and the constraints that influence its development.

Through requirements gathering and analysis, the system's core behaviours are identified and refined. This includes defining key stakeholders, establishing functional and non-functional requirements, and modelling system interactions through use cases and data flows.

The purpose of this chapter is to move from theoretical understanding to practical system definition, ensuring that the design and implementation stages are grounded in clearly defined objectives, user needs, and technical considerations.

3.1 System Overview

FocusFinder is a desktop-based productivity support system designed to help users stay engaged with computer-based work and improve their focus over time. It does this by observing how the user works and responding with supportive, adaptive feedback. From the user's perspective, the system acts as a background assistant rather than a tool that requires constant interaction. Once running, it quietly monitors activity, identifies patterns associated with sustained work or distraction, and provides gentle prompts to encourage a return to productive activity with minimal disruption.

The system is particularly aimed at users who experience difficulty with attention regulation, frequent task switching, or maintaining focus over extended periods. This includes users with ADHD or executive dysfunction, although the system is not limited to this group. In practice, the system provides two main forms of support. The first is immediate support through reminder pop-ups, which appear at regular intervals to prompt reflection on current activity and, where appropriate, encourage re-engagement with work. The second is reflective support through dashboards, summaries, data visualisations, and reports, which allow users to review their behaviour over time and better understand their long-term focus patterns.

A key feature of the system is that it does not rely on a fixed set of repetitive reminders. Traditional productivity tools often deliver identical prompts at predetermined intervals, regardless of how the user responds. In contrast, FocusFinder adjusts its behaviour over time based on user interaction. As the user continues to work, the system records how reminders are received, for example, whether they are acted upon, ignored, or postponed, and uses this history to influence future behaviour. This means that reminders can vary not

only in wording but also in whether they appear, how strongly they are presented, and when they are delivered. Over time, this allows the system to move away from a one-size-fits-all approach and towards a more personalised and responsive support experience.

Rather than acting as a single opaque process, the system can be understood as a set of cooperating elements that observe behaviour, interpret it, and respond appropriately. These elements form a continuous feedback cycle that evolves with the user's interactions over time.

At a high level, the system operates through four main stages:

- **Monitoring:**
  User activity is observed in the background, including application usage, task switching, and periods of inactivity.
- **Interpretation:**
  This activity is analysed to identify patterns such as sustained focus or potential distraction.
- **Decision and Feedback:**
  The system determines whether an intervention is appropriate and selects a suitable reminder.
- **Presentation:**
  Feedback is delivered to the user through prompts, summaries, and visual insights.

These stages are not isolated. Instead, they operate as part of a continuous loop, where each action is informed by previous behaviour and responses. When the user interacts with a reminder, whether by acting on it, dismissing it, or ignoring it, this outcome influences future behaviour, allowing the system to gradually adapt over time.

All information is stored locally on the user's device, ensuring that user activity remains private while still allowing the system to maintain an evolving understanding of behaviour.

In essence, the system operates as a loop:

Monitor → Interpret → Decide → Notify → Observe Response → Improve → (back to Monitor)

where each cycle refines future behaviour based on previous interactions.

3.2 Requirements Gathering

This section outlines the process for identifying and defining the requirements for the Focus Finder system. Building on the research insights presented in the previous chapter, a combination of user-focused and analytical approaches was used to ensure that the system addresses real-world needs while remaining technically feasible.

3.2.1. Key Stakeholders

To support a user-first design approach, it is necessary to identify the groups that influence the system's goals, constraints, and usability requirements. The key stakeholders identified for this project include:

- **Primary Users:**
  Students, professionals, and individuals who experience difficulty with attention, task initiation, or maintaining focus. This includes users with ADHD or executive dysfunction. These users require a system that is supportive, non-judgmental, simple to use, and adaptable to individual working styles.
- **Other Users:**
  Individuals without ADHD who still benefit from assistance in maintaining focus. Their needs align with minimal intrusiveness, practical reminders, and clear productivity insights.
- **Developer / System Maintainer (myself):**
  Responsible for implementing system functionality and ensuring that adaptive behaviour operates reliably. This stakeholder is also concerned with feasibility, system performance, and long-term maintainability.
- **Accessibility and Ethical Considerations:**
  While not a direct stakeholder group, accessibility principles and ethical considerations significantly influence system design. In particular, the system must account for neurodiverse users and ensure that sensitive behavioural data is handled privately and securely.

Together, these stakeholders help define how the system should behave and what constraints it must operate within.

3.2.2. Requirements Collection Methods

To ensure that the system is grounded in real user needs while remaining technically feasible, a combination of qualitative and analytical methods was used to gather initial requirements. These methods were selected to balance direct user input, personal insight, and evaluation of existing solutions.

Interviews and Informal Conversations

Semi-structured conversations were conducted with individuals who experience challenges related to focus, attention regulation, and task switching. This included peers in a college environment, individuals with diagnosed ADHD, and Special Needs Assistants (SNAs) who work closely with students facing attention-related difficulties.

These discussions provided insight into:

- frustrations with existing productivity tools
- preferences around tone and delivery of feedback
- negative experiences with intrusive or overly rigid systems
- The importance of supportive, non-judgmental interaction

This method was chosen because it allows flexible exploration of user experiences, particularly emotional responses to feedback, which are difficult to capture with purely quantitative approaches.

User Survey

To complement qualitative insights, a short questionnaire was distributed to a small group of potential users. The survey focused on identifying common focus-related challenges, preferences regarding reminder frequency, and attitudes towards productivity-tracking tools.

The survey helped to:

- validate recurring themes identified in interviews
- Identify common patterns across a broader group
- confirm demand for adaptive rather than static reminder systems

This method provided lightweight quantitative support for design decisions and ensured that findings were not based solely on anecdotal input.

Observation and Personal Experience

The project was also informed by my personal lived experience with ADHD, combined with long-term use of productivity and focus-support tools. This provided continuous observational insight into how focus fluctuates over time and how different types of interventions affect behaviour.

This approach contributed to identifying:

- The difference between supportive and intrusive system behaviour

- moments where reminders are helpful versus disruptive
- The importance of timing and tone in feedback delivery
- limitations of existing tools in adapting to user needs

While subjective, this perspective was used to complement other methods and was cross-referenced with user feedback to reduce bias.

Analysis of Existing Tools

A review of existing productivity and focus-support applications was conducted, including tools such as RescueTime, Habitica, Forest, Freedom and Cold Turkey.

This analysis focused on:

- What types of data are tracked, and how it is presented
- whether systems adapt to user behaviour over time
- How reminders and feedback mechanisms are implemented
- usability and accessibility considerations

This process helped identify gaps in current solutions, particularly the lack of real-time adaptive feedback and limited personalisation, which directly informed the direction of this project.

Technical Feasibility Review

A technical feasibility review was conducted to ensure that the proposed system behaviours could be realistically implemented in a desktop environment.

This included investigating:

- How application usage and idle time can be monitored
- What operating system-level data is accessible
- The limitations of tracking browser activity
- constraints on local data storage

This step ensured that the final system design remained achievable within the project scope and available technologies.

3.3 Requirements Analysis

This section translates the previously gathered requirements into an initial system model. It defines the key actors involved in system interaction, identifies the primary use cases, and analyses the overall behavioural structure required for the system to operate effectively.

3.3.1 System Actors

Based on the identified requirements, the following actors were defined. These represent entities that interact with or influence the system's behaviour.

- **User:**
  The primary actor of the system. The user initiates monitoring sessions, receives reminders, interacts with notifications, and reviews activity data through the dashboard.

- **Operating System Environment (External Actor):**
  Provides the system with access to application activity, window focus changes, and idle time signals. This actor enables the system to observe user behaviour without requiring manual input.

In addition to external actors, the system relies on several internal functional elements which, while not actors in the traditional sense, are important in understanding system behaviour:

- **Activity Monitoring:**
  Responsible for continuously observing user activity in the background.
- **Activity Interpretation:**
  Processes observed activity and classifies it into meaningful behavioural categories such as focused, neutral, or distracting.
- **Adaptive Feedback Mechanism:**
  Determines how and when reminders are delivered, taking into account timing rules and prior user responses.
- **Local Data Store:**
  Maintains a record of activity, classifications, and reminder outcomes, enabling the system to operate consistently over time.

3.3.2 Identified Use Cases

Based on the gathered requirements, several high-level use cases were identified. These describe the key interactions between the user and the system, as well as the system's core operational behaviours.

- **Start Monitoring Session:**
  The user enables activity tracking, allowing the system to observe behaviour in the background.
- **Track and Classify Activity:**
  The system records application usage and inactivity, and assigns behavioural classifications based on observed patterns.
- **Deliver Reminder:**
  The system presents reminder notifications at defined intervals, with behaviour influenced by previous user interactions and system context.
- **Respond to Reminder:**
  The user may acknowledge, ignore, or postpone reminders. These responses are recorded and used to influence future system behaviour.
- **View Dashboard and Adjust Preferences:**
  The user reviews activity summaries and modifies settings such as reminder style and notification frequency.

These use cases define the system's functional scope and lay the foundation for further modelling, including the use case diagram presented in the following subsection.

3.3.3 Analysis of System Behaviour

By drawing on information gathered from the identified actors and use cases, the system can be understood as operating through a continuous cycle of interaction. The user initiates system activity, after which the system operates largely autonomously, observing behaviour, delivering reminders, and adapting over time based on user responses.

A key insight from this analysis is that the system does not rely on a single interaction point but instead maintains an ongoing relationship with the user throughout operation. The majority of system behaviour occurs in the background, with user interaction occurring primarily through reminder notifications and dashboard reviews. This reinforces the need for low intrusiveness and highlights the importance of timing and adaptability in delivering feedback.

Additionally, the system's reliance on the operating system as its primary data source limits what behaviour can be observed and how accurately it can be interpreted. These limitations directly influence the design of the monitoring and interpretation processes.

Overall, this analysis establishes the system as a continuous feedback loop, in which user behaviour influences system decisions, and those decisions in turn influence future user behaviour.

3.4 Logical Architecture of System

This section defines the logical structure of the Focus Finder system, describing how its core components interact to support continuous monitoring, interpretation, and adaptive feedback. Building on the system model established in the previous section, it outlines how the system is organised at a conceptual level.

3.4.1 System structure

The system's logical architecture is based on a small set of cooperating components, each with a clearly defined responsibility within the overall monitoring and feedback loop. These components operate together to support continuous observation, interpretation, and adaptive response.

At a conceptual level, the system is organised into the following core elements:

- **User Interface Layer:**
  Provides the primary point of interaction for the user. It allows users to start or stop monitoring, review focus patterns through dashboards, and adjust system preferences such as reminder behaviour.
- **Activity Monitoring Component:**
  Responsible for observing user activity in the background, including application usage, task switching, and periods of inactivity. This component forms the foundation of the system's behavioural data.
- **Activity Interpretation Component:**
  Processes monitored activity and classifies it into meaningful categories such as focused, neutral, or distracting behaviour. This classification provides the context required for decision-making.
- **Adaptive Feedback Component:**
  Determines how the system responds to user behaviour by selecting whether and how reminders are delivered. This component incorporates previous user interactions to support adaptive behaviour over time.
- **Local Data Store:**
  Maintains a persistent record of activity data, classifications, reminder outcomes, and user preferences. It acts as a shared source of information for all other components.

These components are not independent, but instead operate as part of a continuous flow of information. Activity is first observed, then interpreted, before being used to inform feedback decisions. User responses are recorded and fed back into the system, allowing future behaviour to be adjusted accordingly.

This structure reflects the system's core operational loop:
Monitor → Interpret → Decide → Notify → Observe Response →Adapt
which forms the basis of the system's adaptive functionality.

3.4.2 Data Flow

Because the system must adapt in real time, there must be a continuous flow of information between its core components, supporting the monitoring, interpretation, and adaptive feedback cycle described earlier.

At a high level, activity data is received from the operating system, including application usage, window focus changes, and periods of inactivity. The activity monitoring component processes these signals, recording and storing raw activity data in the local data store.

This stored data is then used by the activity interpretation and learning component to classify behaviour into meaningful categories, such as focused, neutral, or distracting. These classifications, together with historical data and user preferences, provide the context required for system decision-making.

The adaptive feedback component uses this information to determine how reminders should be delivered. While reminders are issued according to system timing rules, their tone, intensity, and presentation are influenced by both recent behaviour and previous user responses. When a reminder is presented, the user may acknowledge, dismiss, or snooze it, and this response is recorded in the system's behavioural history.

The user interface uses the same data to present summaries such as focus trends, reminder history, and preferences, and allows the user to modify system settings. These updates are incorporated into future system behaviour, ensuring that the system continues to adapt over time.

Overall, this flow of information forms a closed feedback loop in which user behaviour influences system decisions, and those decisions, in turn, influence future user behaviour.

The data flow diagram in Figure 7 illustrates this process.

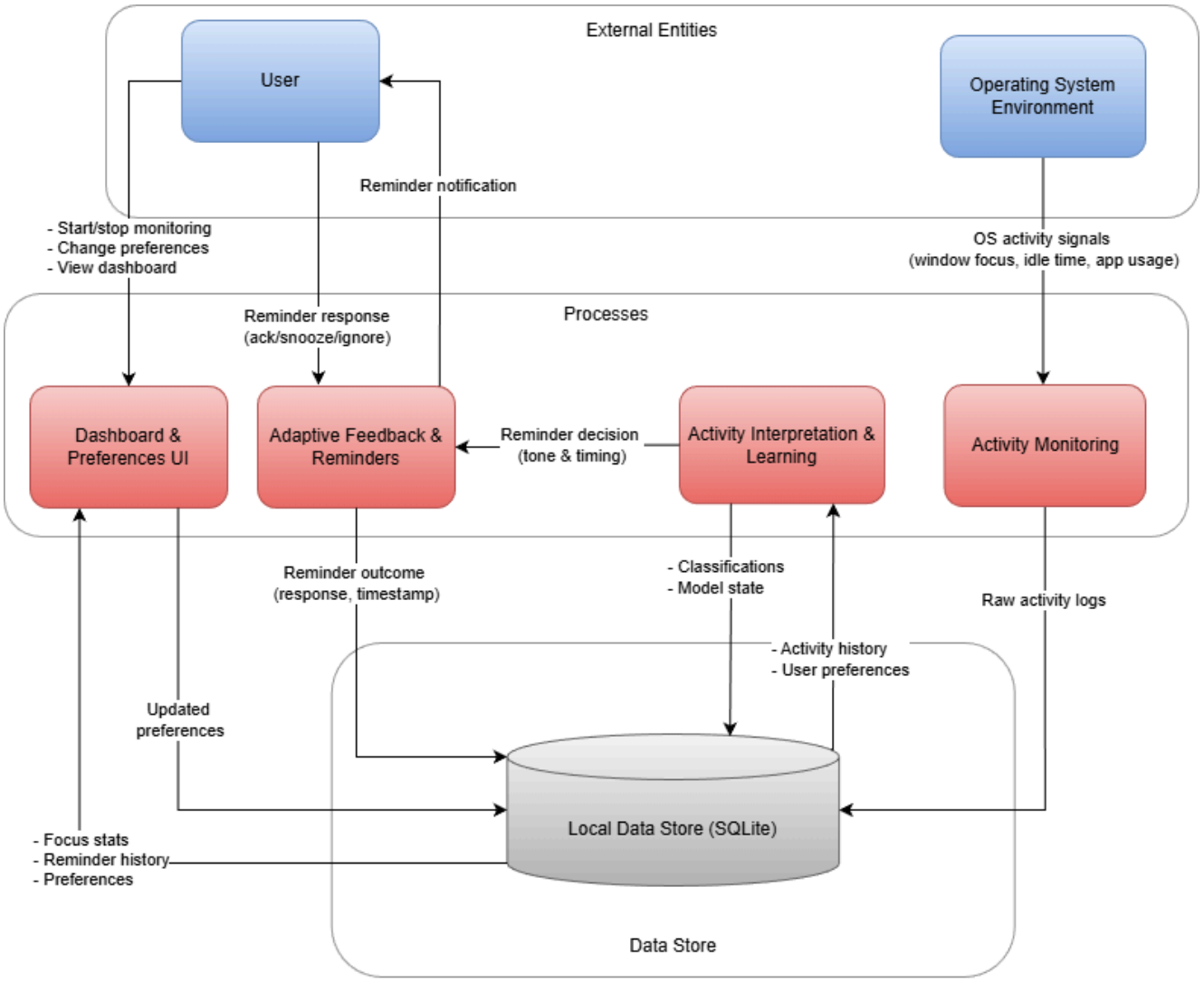


Figure 7: Data Flow Diagram of the FocusFinder System

3.4.2 System Behaviour

The behaviour of the system over time can be understood as a sequence of states that reflect its monitoring and feedback process. While the data flow describes how information moves through the system, the behavioural model focuses on how the system progresses through different stages of operation.

The system begins in an idle state and transitions to an active state when the user enables monitoring. In this active state, the system continuously observes user activity and processes new behavioural data as it becomes available.

As activity is recorded, the system progresses through a classification stage, where behaviour is interpreted and categorised. This classification informs the subsequent decision stage, in which the system determines whether to deliver a reminder. Although reminders are governed by system timing rules, this decision is influenced by behavioural context and previous user interactions.

If a reminder is delivered, the system enters a notification state, followed by a waiting state that waits for user interaction. The user may acknowledge, dismiss, or snooze the reminder, after which the system processes the response and updates its internal state accordingly.

Following this, the system returns to its active monitoring state, allowing the cycle to repeat. This loop continues until the user stops monitoring, at which point the system transitions to a stopped state.

A visual representation of these state transitions is shown in Figure 8.

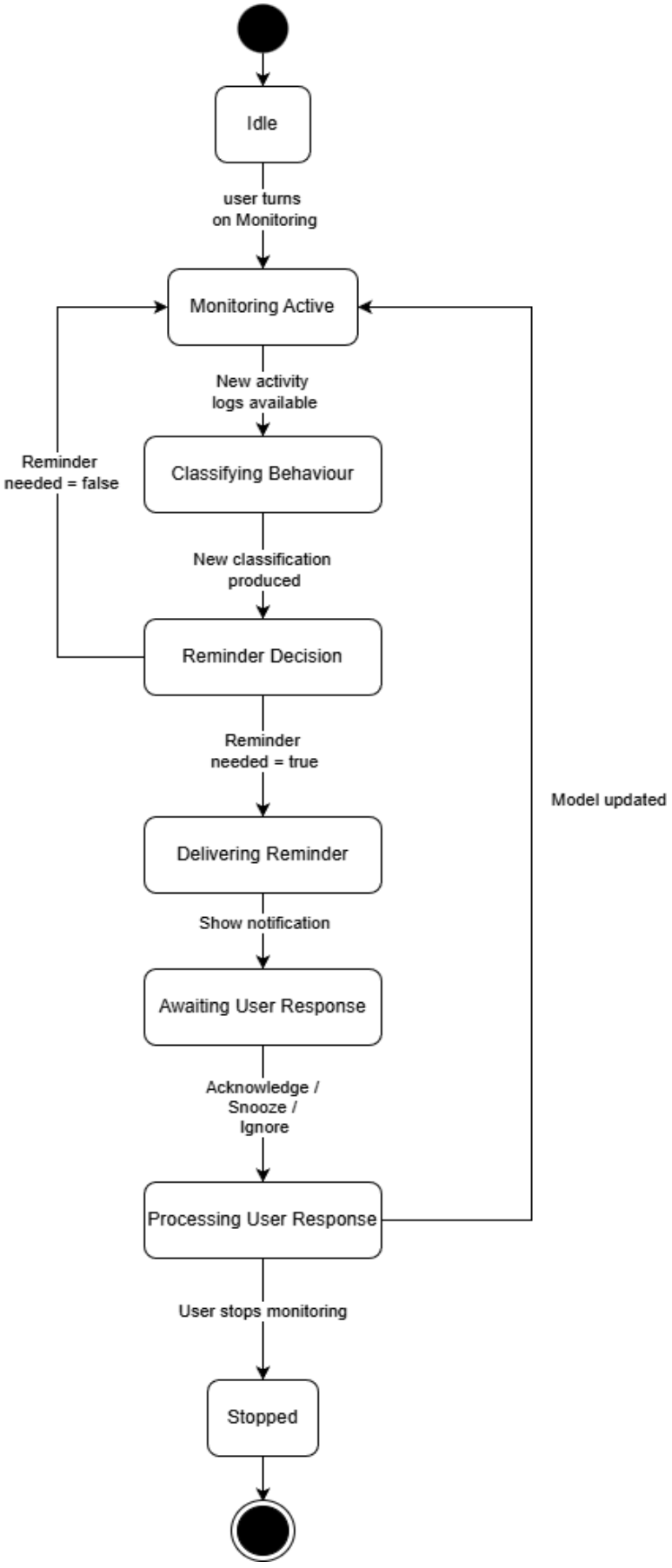


Figure 7: State Diagram of the FocusFinder System

3.5 Initial System Specification

The requirements identified during the earlier stages of analysis were formalised into an initial system specification.
This specification defines the core functional and non-functional expectations of the system and establishes the basis for subsequent design decisions.

3.5.1 Functional requirements

ID	Requirement Description
FR 1	The system shall allow the user to start and stop activity monitoring.
FR 2	The system shall track active applications, task switches, and idle periods while monitoring is enabled.
FR 3	The system shall store recorded activity data in a local data store.
FR 4	The system shall classify recent activity as focused, neutral, or distracting.
FR 5	The system shall infer potential distraction from observed behavioural signals such as application usage, task switching, and idle time.
FR 6	The system shall select and deliver reminders based on current behavioural context, timing rules, and past user responses.
FR 7	The system shall log the user's response to each reminder (acknowledge, snooze, ignore).
FR 8	The system shall update its internal model based on the outcomes of reminders.
FR 9	The system shall present dashboards and reports detailing focus patterns, distraction trends, reminder history, and related behavioural summaries.

3.5.2 Non-functional requirements

ID	Requirement Description
NFR 1	All data shall be stored locally on the user's device. No behavioural data shall be transmitted externally.
NFR 2	Monitoring and classification shall run unobtrusively without noticeably impacting system performance.
NFR 3	The system shall use minimal CPU and memory resources during operation.
NFR 4	The user interface shall be simple and easy to navigate, requiring no training.
NFR 5	The system shall adapt reminder behaviour over time based on user-specific responses.
NFR 6	The system shall operate reliably and avoid corrupting data in the case of crashes or unexpected shutdowns.
NFR 7	The system shall use technologies appropriate to the target desktop platform (Windows) and feasible within the project constraints.
NFR 8	Notifications and UI text shall use supportive, non-judgemental language appropriate for users with ADHD or executive dysfunction.

The requirements defined above directly inform the system's architectural design. In particular, the need for continuous background monitoring, local data storage, and adaptive feedback behaviour leads to a modular structure in which monitoring, interpretation, and feedback responsibilities are clearly separated.

3.6 Conclusions

This chapter establishes the analytical foundations on which the FocusFinder system is built, defining how it should operate, who it is designed for, and the constraints that shape its development. Through a structured process of requirements gathering and analysis, the system's core behaviour was identified and refined prior to implementation.

The findings in this chapter validate the initial design direction, particularly the need for a low-intrusion, adaptive system that supports users with varying attention needs. Insights from user feedback and behavioural analysis highlighted the importance of timing, tone, and non-intrusive interaction, reinforcing the need for a system that responds to user behaviour rather than relying on rigid or static approaches.

The insights gained through requirements gathering were then translated into a defined set of actors and use cases, establishing how the system should behave and what interactions it must support. This process informed the design of a continuous feedback loop model, in which user behaviour influences system decisions and those decisions, in turn, shape future behaviour.

The logical architecture and data flow representations build on this foundation by illustrating how the system's components interact to support this behaviour while maintaining a strong emphasis on privacy, security, and local data processing.

Finally, the definition of functional and non-functional requirements provided a clear specification that guided the system's development. While the implementation phase introduced refinements, the core principles and structure defined in this chapter remained central to the final system.

Overall, this chapter establishes a clear and justified foundation for the design and implementation decisions presented in the following chapter.

4. System Design

4.1 Introduction

This chapter presents the complete internal design of the Focus Finder system, detailing how the application is structured and how each component contributes to its overall behaviour.

The aim is to take the system requirements discussed earlier in Chapter 3 and form a coherent architecture, supported by diagrams that illustrate data flow and the responsibilities of each component relative to one another.

4.2. Software Methodology

The development of this system follows a component-based, layered architectural approach used to structure the implementation. The system is organised into distinct functional components, including activity monitoring, behaviour interpretation, adaptive feedback, and data management, each with a clearly defined responsibility as established in the system analysis. This approach allows individual parts of the system to be implemented and tested in isolation, providing a clear and manageable path for development.

Building on this structure, an Agile-inspired development approach was used to implement the system incrementally. This approach was adapted specifically for a single-developer project, where formal Agile practices such as sprint ceremonies and team coordination were unnecessary. Instead, development was organised into small, focused iterations, each targeting a specific component or feature.

Functionality was introduced in stages, beginning with core features such as activity monitoring and data logging, before progressing to behaviour classification and adaptive feedback. This incremental process allowed issues to be identified early and made it easier to track and manage development progress. Working within clearly defined scopes, it helped maintain focus and reduce the risk of unnecessary deviation during implementation.

The iterative approach also supported ongoing refinement of system behaviour as components were integrated. Adjustments to feedback timing, interaction design, and system responsiveness were made throughout development, particularly in areas where small changes had a noticeable impact on user experience.

4.3 Project Planning

Project development was planned in line with the component-based and iterative methodology described in the previous section. Development was organised into distinct stages based on individual system components, enabling work to proceed in a controlled, incremental manner.

Component-Based Planning
Development was structured around the system's core components, allowing each to be addressed independently. This approach kept complexity contained within smaller, manageable parts while ensuring a clear flow through the system.

Research and Preparation
Before implementing each component, a brief research and investigation phase was conducted. This involved identifying suitable technologies, including alternatives where initial choices proved unsuitable, understanding implementation constraints, and defining a practical approach to development. This step reduced uncertainty and helped avoid inefficient trial-and-error during implementation.

Time Estimation and Flexibility
Approximate time estimates were assigned to each component based on expected complexity. These served as flexible guidelines rather than fixed deadlines, with additional time allowed for debugging, setbacks, and interruptions. This was particularly important for more complex features, such as behaviour classification and adaptive feedback.

Iterative Adjustment
The planning approach allowed for ongoing adjustment throughout development. As components were implemented and tested, new ideas and improvements could be incorporated without disrupting overall progress. This allowed the system to evolve incrementally while remaining aligned with the project's core requirements and objectives.

4.3.1 Project Schedule

The project schedule was designed to reflect the iterative development approach while accounting for practical academic constraints. Rather than following a strictly sequential structure, tasks were organised into overlapping phases to allow development, testing, and documentation to progress in parallel.

Key considerations in the schedule include:

- **Academic Constraints:**
  Examination periods, interim report submission, and project presentation deadlines were factored into the timeline to ensure a realistic workload distribution.

- **Documentation Time:**
  Dedicated time was allocated for report writing and revision, allowing documentation to be developed alongside implementation rather than left to the final stages.

- **Component-Based Progression:**
  Development tasks were structured around core system components, with early stages focusing on foundational functionality and later stages supporting integration and refinement.

- **Allowance for Iteration and setback:**
  Additional time was included for debugging, unexpected challenges, and iterative improvements, particularly for more complex features such as adaptive feedback.

This approach ensured a balance between structured planning and flexibility, enabling consistent progress while allowing the system to evolve during development. The full project timeline is illustrated in Figure 8.

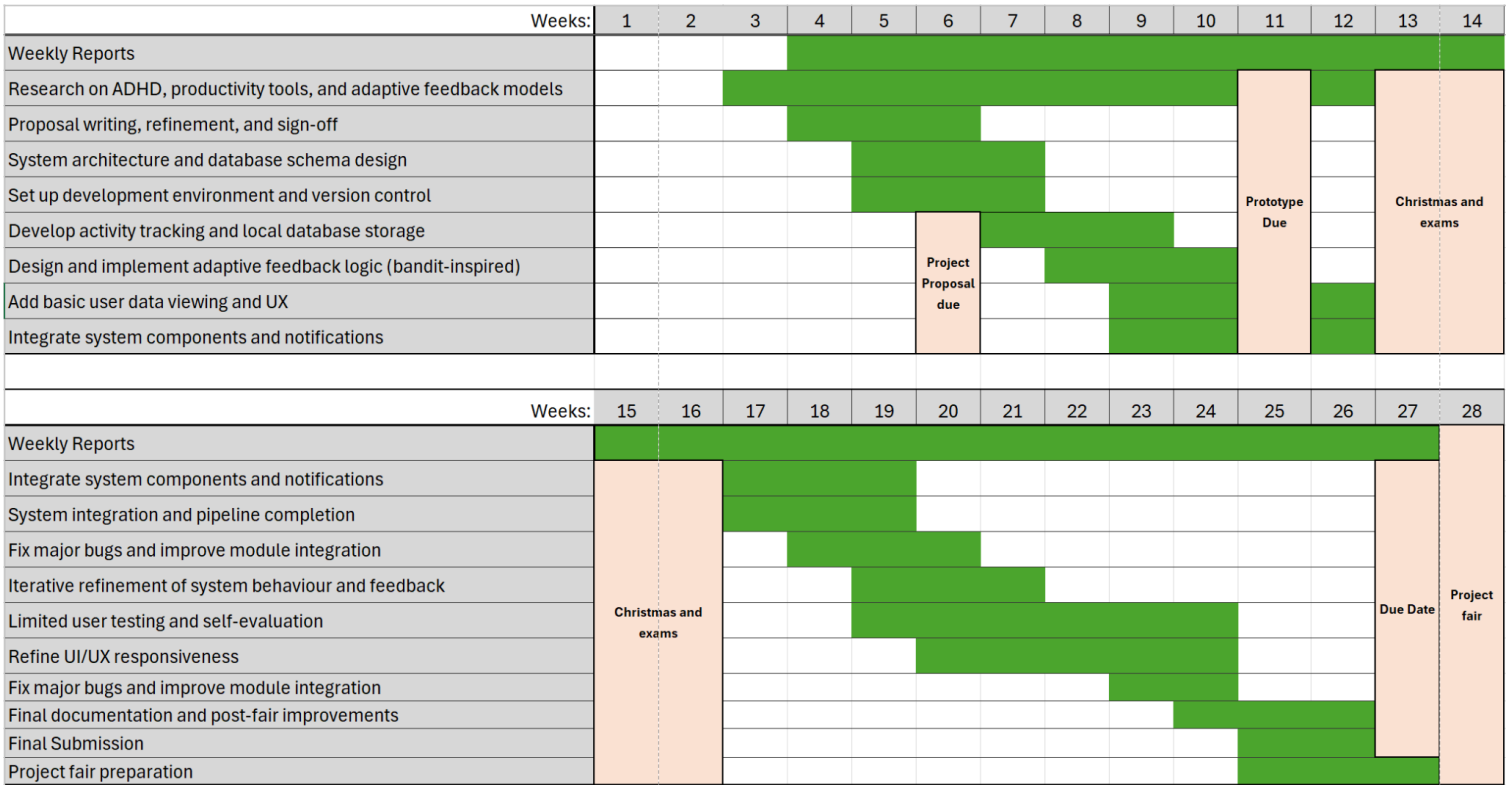


Figure 8: Gantt chart of project schedule

4.4. Overview of System

This section presents the overall design of the Focus Finder system, outlining how its components are structured and interact within the local environment. It provides a high-level overview of the system before examining its technical architecture and how it fulfils the requirements identified in the previous chapter.

4.4.1 Technical Architecture

The technical architecture of the Focus Finder system is shown in Figure 9. The system is deployed entirely on the user's local machine and is structured into three primary layers: the interface layer, the service layer, and the data and operating system layer.

Each layer has a clearly defined role within the system, and their interactions form a continuous processing pipeline that supports monitoring, adaptive decision-making, and feedback delivery.

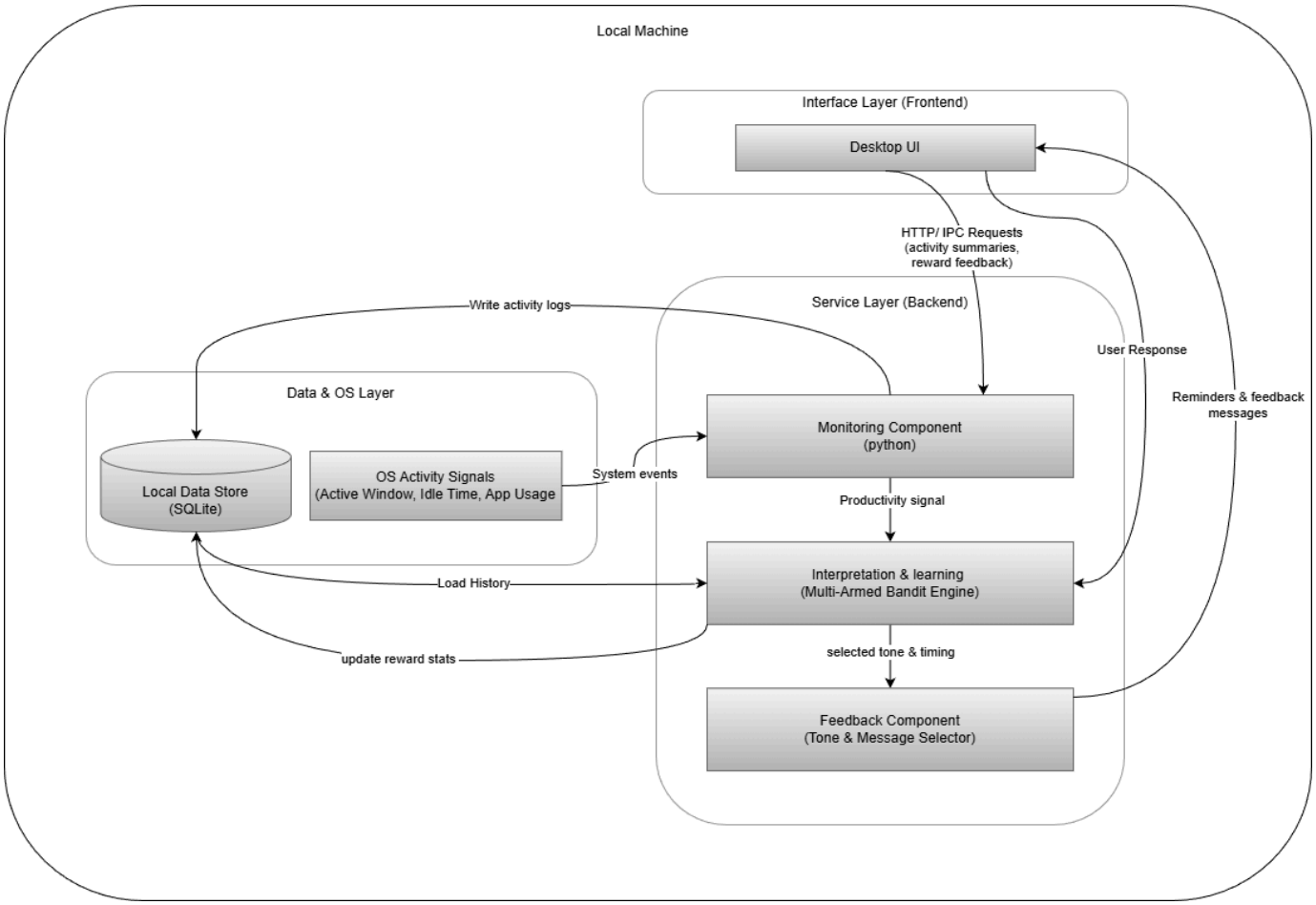


Figure 9: Technical Architecture Diagram

Interface Layer (Front End)

As shown in Figure 9, the interface layer provides the primary point of interaction between the user and the system. It consists of a desktop application that allows users to start and stop monitoring, view activity summaries, and configure system behaviour such as reminder tone, sensitivity, and quiet hours.

The interface communicates with the service layer via lightweight requests to retrieve processed activity data and submit user preferences. This separation ensures that user interaction remains independent from background processing.

Service Layer (Monitoring and Learning Engine)

The service layer operates as a background process and is responsible for the system's core functionality. Figure 9 shows that it is composed of three main components:

- **Monitoring Component:**
  Responsible for collecting activity data from the operating system, including active window focus, application usage, and idle time.

- **Interpretation and Learning Component:**
  Processes collected activity data and maintains the adaptive model that determines system behaviour. This component incorporates an adaptive decision-making approach, allowing feedback strategies to be refined over time based on observed user interaction.

- **Feedback Component:**
  Selects and delivers reminder messages, including tone and timing, based on decisions produced by the learning component.

Data flows sequentially through these components, with user responses to reminders fed back into the system to influence future behaviour.

Data and Operating System Layer

As shown in Figure 9, the data and operating system layer provides both input signals and persistent storage. Activity data is obtained directly from the operating system, including window focus changes and idle time detection.

All system data is stored locally using a lightweight database, including activity logs, user preferences, and reminder outcomes. This supports consistent system behaviour while ensuring that all data remains on the user’s device.

The data flow represented in Figure 9 reflects the logical model previously described in Figure 7, with system components now expressed as concrete implementation layers and services.

Layer Interaction and Data Flow

The interaction between layers, illustrated in Figure 9, forms a continuous processing pipeline. Activity signals are collected from the operating system and passed to the service layer for processing. The resulting decisions are delivered to the user through the interface layer.

User responses are returned to the service layer, where they influence subsequent system behaviour. This closed-loop structure enables the system to adapt over time while maintaining a clear separation between monitoring, decision-making, and user interaction.

4.4.2 Fulfilment of System Requirements

The system design presented in subsection 4.4.1 directly addresses the functional and non-functional requirements identified during system analysis. These requirements are fulfilled through the system's layered architecture, with each requirement mapped to a specific component or interaction. This mapping is summarised in Tables 3 and 4.

ID	Requirement	Design Implementation
FR 1	**Monitoring Control**	Implemented through the interface layer, allowing the user to start and stop monitoring sessions via the desktop application.
FR 2	**Activity Tracking**	Handled by the monitoring component within the service layer, which collects application usage, task switching, and idle time from the operating system.
FR 3	**Local Data Storage**	Achieved through the data layer, where all activity data is stored in a local database.
FR 4 & FR 5	**Activity Classification**	Performed by the interpretation and learning component, which processes raw activity data and categorises behaviour into focused, neutral, or distracting states
FR 6	**Adaptive Reminder Delivery**	Managed by the feedback component, which selects and delivers reminders based on behavioural context and prior user responses.
FR 7	**Response Logging**	User interactions with reminders are captured and stored within the data layer, forming part of the system's behavioural history.
FR 8	**Model Adaptation**	Supported by the interpretation and learning component, which updates its internal model based on recorded reminder outcomes.
FR 9	**Dashboard and Reporting**	Presented within the interface layer, where processed activity data is presented as summaries and visual insights.

Table 3: Mapping of Functional Requirements to System Design

ID	Requirement	Design Implementation
NFR 1	**Privacy and Local Processing**	The system operates entirely on the user's device, with all data stored locally and no external transmission.
NFR 2 & NFR 3	**Low Intrusion and Performance**	Background monitoring is handled within the service layer, ensuring minimal impact on system performance and unobtrusive operation.
NFR 4	**Usability**	The interface layer provides a simple and accessible user experience, with clear controls and minimal interaction requirements.
NFR 5	**Adaptive Behaviour**	Implemented through the learning component, enabling the system to adjust reminder behaviour over time.
NFR 6	**Reliability**	Persistent local storage ensures data integrity and continuity across sessions.
NFR 7	**Platform Compatibility**	The system is designed specifically for a Windows desktop environment, aligning with the project constraints.
NFR 8	**Supportive Feedback Design**	The feedback component delivers reminders using configurable tone and presentation, supporting non-judgmental interaction.

Table 4: Mapping of Non-Functional Requirements to System Design

4.5 Additional Design Representations

This section presents supplementary design representations that extend the previously described system architecture. These artefacts illustrate how the system is intended to support user interaction and how its core components coordinate during runtime, providing a more detailed view of system behaviour prior to implementation.

4.5.1 User Interface Wireframes

Figures 10–12 present the initial mockups for the Focus Finder interface, including the main dashboard, daily statistics view, and reminder interaction design. These designs illustrate how activity data and system feedback are presented to the user, including focus summaries, activity timelines, and progress indicators.

The interface is designed to prioritise clarity and low cognitive load, allowing users to quickly understand their current focus state and interact with the system without disruption.

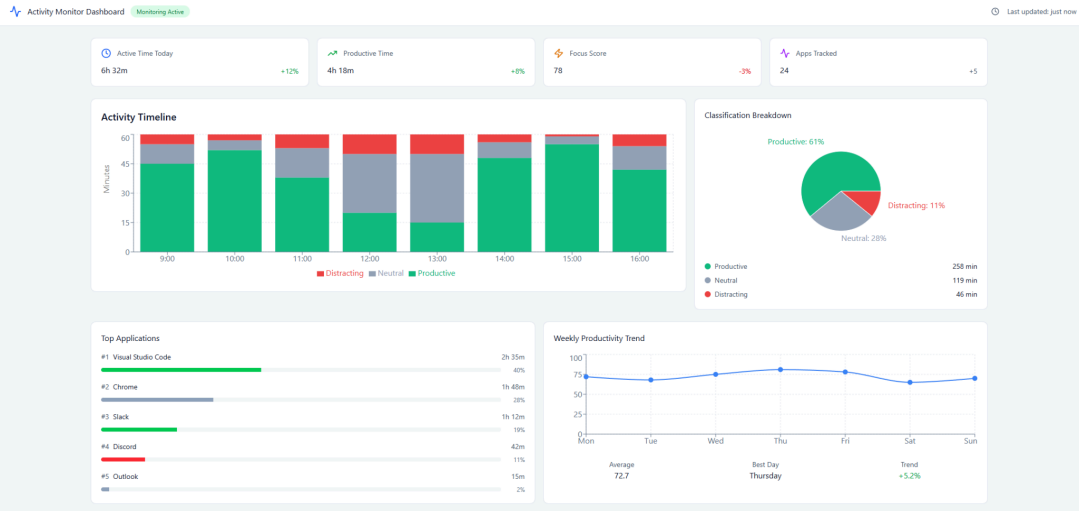


Figure 10: Dashboard layout mockup

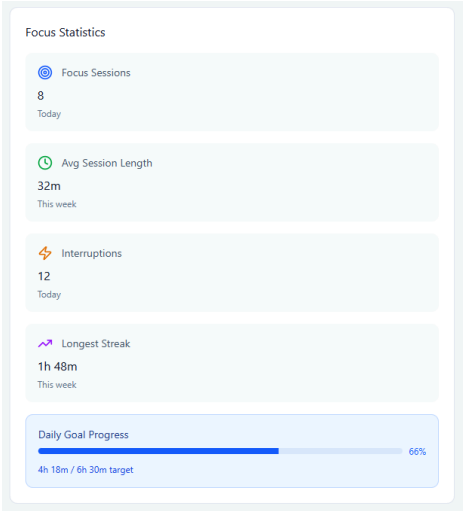


Figure 11: Daily statistics mockup

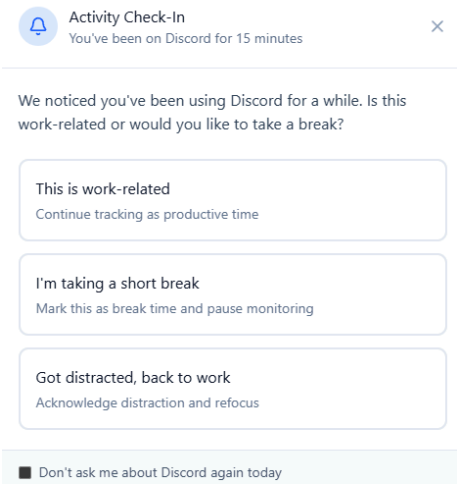


Figure 12: Pop-Up Reminder mockup

4.5.2 Sequence Diagram

Figure 13 illustrates the runtime interaction between system components during a monitoring session. The diagram shows how user actions, system monitoring, interpretation, and feedback components interact over time.

In particular, it highlights the continuous monitoring loop and the conditional delivery of reminders based on system evaluation. It also demonstrates how user responses are captured and fed back into the system to influence subsequent behaviour.

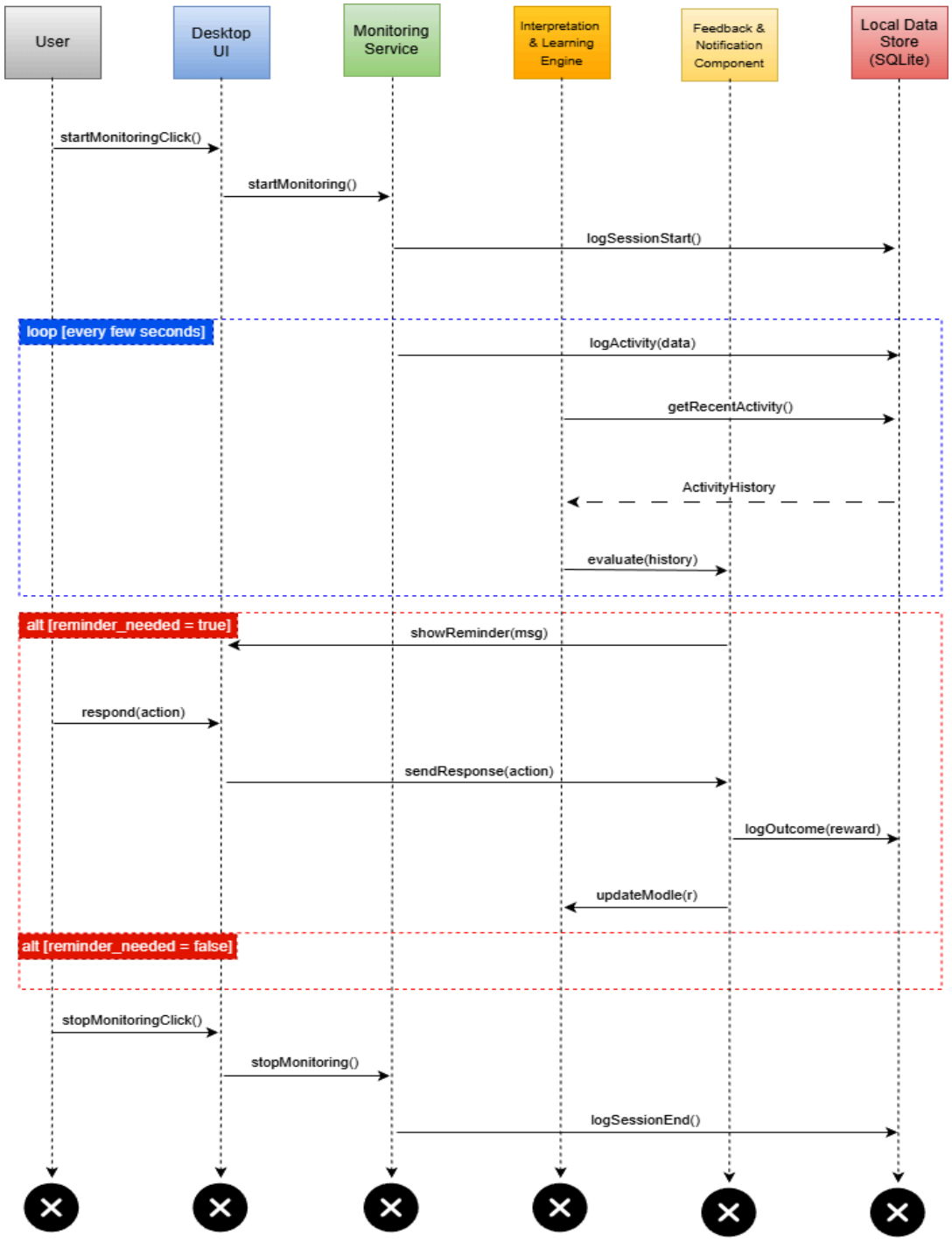


Figure 13: Sequence Diagram

4.5 Conclusions

This section has presented the complete design of the Focus Finder system, translating the requirements identified during system analysis into a structured, implementable architecture. The system is organised as a layered, component-based design that separates user interaction, background processing, and data management to support modularity and maintainability.

The technical architecture demonstrates how monitoring, interpretation, and feedback components cooperate within a continuous processing pipeline, enabling adaptive behaviour while maintaining low system intrusion. The inclusion of supplementary design representations further illustrates how the system is intended to function at the interface level and during runtime interactions.

Overall, the design provides a practical and flexible foundation for implementation, ensuring that the system remains efficient, privacy-preserving, and adaptable to user behaviour over time. The next chapter builds upon this design by detailing the development and implementation of the system components.

5. System Development

5.1. Introduction

This chapter presents the development of the Focus Finder system, describing how the research, analysis, and design decisions from previous chapters were translated into a working application. While Chapter 2 explored relevant technologies and evidence-based approaches, Chapter 3 defined the system requirements, and Chapter 4 outlined the system architecture and design. This chapter builds on those foundations, documenting the practical implementation of the system.

The chapter is structured around the implementation of the system's core components, including monitoring and activity capture, activity classification, database design, adaptive reminder generation, and user interface integration. These components are examined individually before being brought together as a fully integrated system.

5.2 Component Development

The Focus Finder system was developed as a desktop application combining an Electron-based user interface with a Python backend responsible for monitoring, data processing, and adaptive feedback. Figure 14 illustrates the implemented technology stack and runtime data flow, showing how user activity is captured, processed, and used to drive personalised interventions.

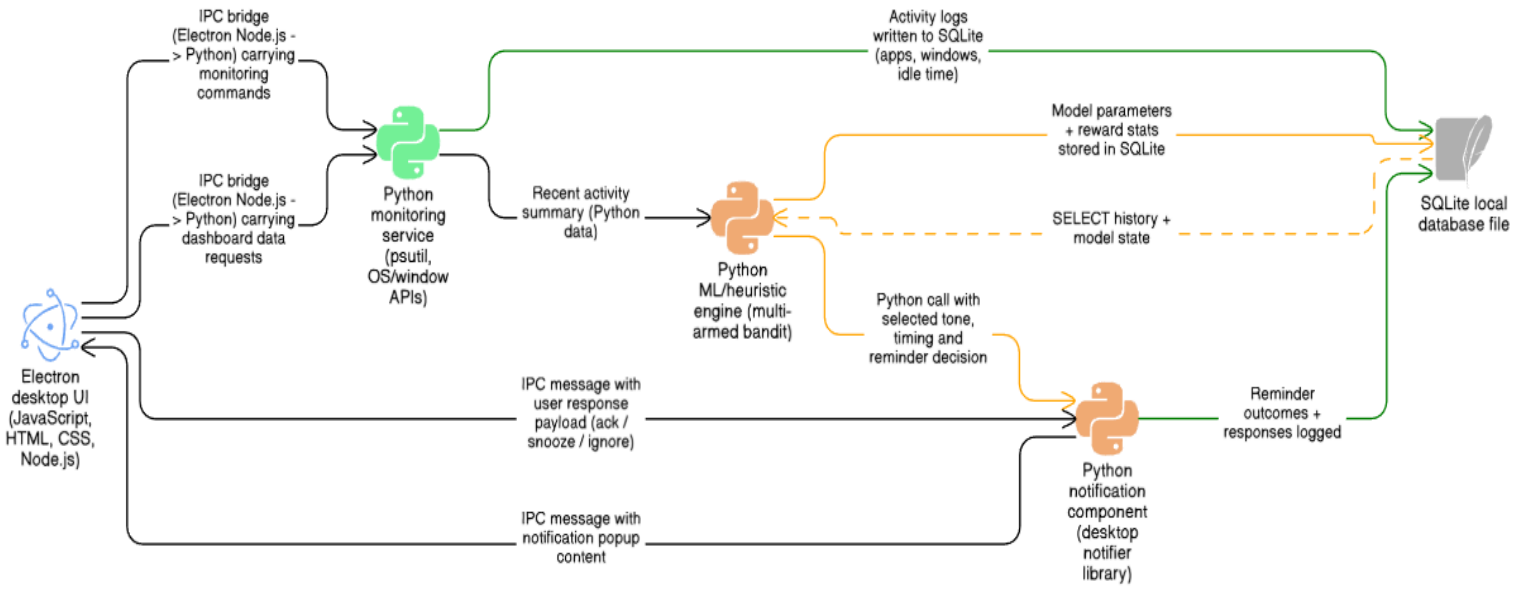


Figure 14: Technology Stack and Runtime Data Flow Diagram

5.2.1 Monitoring and Activity Capture

The monitoring and activity capture subsystem is responsible for continuously collecting user activity data from the operating system. This implementation reflects the requirement for passive, low-intrusion monitoring identified during system analysis (Chapter 3), and supports the local-first processing model defined in the system design (Chapter 4).

Core Modules

The monitoring subsystem is composed of three primary modules located within the "`monitoring/`" package:

- **ActivityTracker** ("`activity_tracker.py`")
    - Central coordination component
    - Implements the monitoring loop
    - Responsible for constructing and recording activity events
- **IdleDetector** ("`idle_detector.py`")
    - Uses "`pynput`" to listen for mouse and keyboard input
    - Tracks the time elapsed since the last user interaction
    - Provides idle duration via "`get_idle_duration()`"
- **WindowTracker** ("`window_tracker.py`")
    - Uses "`pygetwindow`" to identify the active window
    - Extracts the application name and window title
    - Tracks the window switching behaviour

These components are initialised and connected in "`main.py`", where the monitoring loop runs in a background thread alongside the API service.

Captured Activity Data

Each monitoring cycle produces a structured activity event with the following fields:

- **Application Name**
    - Derived from the active window process
- **Window Title**
    - Provides contextual detail (e.g. browser tab content or document name)

- **Idle Time (seconds)**
    - Calculated based on the time since the last input event
- **Activity State**
    - Indicates whether the user is actively interacting with the system

The structure of each activity event is defined when persisting data via

```
self.db_manager.store_activity_event(
    app_name=app_name,
    timestamp=timestamp,
    idle_time=idle_time,
    window_title=window_title,
    classification=classification,
    is_active=is_active
)
```

"`DbManager.store_activity_event()`", as shown in Figure 15.

Figure 15: Structure of Recorded Activity Event

This structure ensures that each activity record contains both contextual information (application and window title) and behavioural indicators (idle time, classification, and activity state).

Monitoring Loop Implementation

The monitoring process is controlled by the "`ActivityTracker`" component, which executes a continuous polling loop:

- **Polling Interval:** 60 seconds
- **Execution Flow per Cycle:**
    1. Retrieve idle duration from "`IdleDetector`"
    2. Retrieve active window data from "`WindowTracker`"
    3. Construct an activity event
    4. Pass the event to "`ActivityClassifier`"
    5. Persist event via "`DbManager.store_activity_event()`"

If no active window is detected, the event is recorded as inactive or neutral.

This loop is implemented in "`ActivityTracker.start_monitoring()`" and serves as the central coordination point for the monitoring pipeline.

Role Within the System

The monitoring subsystem forms the entry point of the system's data pipeline:

IdleDetector + WindowTracker → ActivityTracker → ActivityClassifier → Database

By continuously capturing structured activity events, this component enables all downstream functionality, including behavioural classification, dashboard analytics, and adaptive feedback generation.

5.2.2 Activity Classification and Behaviour Interpretation

The activity classification and behaviour interpretation subsystem is responsible for transforming raw monitoring data into meaningful behavioural labels. This implementation reflects the need for interpretable, context-aware feedback identified in the literature review (Chapter 2), particularly in relation to ADHD-sensitive feedback systems, and builds upon the structured data pipeline established in the system design (Chapter 4).

Core Modules

The classification subsystem is implemented across two primary modules within the "`interpretation/`" package:

- **ActivityClassifier** ("`activity_classifier.py`")
    - Core classification engine
    - Assigns each activity event a label: focused, neutral, or distracted
    - Applies rule-based heuristics based on application name and window title
- **ADHDHelper** ("`adhd_helper.py`")
    - Provides ADHD-specific behavioural interpretation
    - Adjusts classifications based on time-of-day patterns
    - Detects behavioural patterns such as hyperfocus, rapid task switching, and task avoidance

These components are invoked during each monitoring cycle, with classification results returned to the monitoring pipeline before being persisted to the database.

Classification Logic

The classification process is implemented within "`ActivityClassifier.classify_activity()`", which evaluates each activity event using a rule-based decision chain.

The classifier operates on the following inputs:

- Application name
- Window title
- Timestamp

The core classification logic is implemented as a prioritised conditional structure, as shown in Figure 16.

```
if self._is_focused_app(app_name, window_title):
    base_classification = 'focused'
elif self._is_distracted_app(app_name, window_title):
    base_classification = 'distracted'
elif self._is_neutral_app(app_name):
    base_classification = 'neutral'
```

Figure 16: Core Classification Decision Logic

This decision chain evaluates application context in a fixed order, ensuring that more specific classifications take priority over general cases.

The classification system is supported by large curated application sets, which group software into three primary categories:

- **Focused applications** (development environments, office tools, research platforms)
- **Distracted applications** (social media, streaming, gaming platforms)
- **Neutral applications** (system utilities, file managers, background tools)

These sets cover a wide range of real-world applications, enabling effective classification without training data and allowing easy extension.

Browser activity is treated as a special case, with the window title used to distinguish productive from distracting use.

Behavioural Interpretation (ADHD Layer)

Following base classification, the result is passed to `"ADHDHelper.adjust_classification_for_time_of_day()"`, which applies ADHD-aware adjustments.

This includes:

- **Morning leniency (6–9 am)**: If the base classification is "distracted," it is downgraded to "neutral" before being persisted.
  (Reduced executive function during early morning hours is a well-documented ADHD symptom. Recording this as a distraction would misrepresent a biological pattern as a behavioural failure and inflate daily distraction scores inaccurately.)
- **Afternoon energy dip (2–4 pm)**: Distracted classifications during this window are preserved unchanged, but the event is logged.
  (The afternoon crash is recognised as a known ADHD attention pattern; the label is kept accurate for analytics while the context is available for downstream components such as reminders.)

In addition to real-time adjustment, the module provides pattern detection functions, including:

- **Hyperfocus detection:** Queries the last 2.5 hours. If 85% or more of intervals are classified as focused, a warning is raised.
- **Rapid task switching detection:** Counts distinct app switches within a 5-minute window. If the count exceeds the threshold (default: 15), it flags a potential state of overwhelm or avoidance.
- **Task avoidance pattern detection:** Examines the last 15 minutes of classifications for an alternating distracted, neutral, distracted cycle. If more than 60% of consecutive transitions follow this pattern, it is flagged as likely task avoidance.

These features extend the system beyond simple per-event labelling, enabling detection of temporally-structured behavioural patterns that are clinically relevant to ADHD but invisible when examining individual events in isolation.

Role Within the System

The classification subsystem forms the interpretation stage of the system's data pipeline:

ActivityTracker → ActivityClassifier → ADHDHelper → Database

By transforming raw activity data into structured behavioural labels, this component enables downstream functionality, including dashboard analytics, ADHD-specific metrics, and adaptive feedback generation.

5.2.3 Data Storage and Database Design

The data storage and database subsystem is responsible for persisting all activity data, behavioural insights, and system interactions in a structured local database. This implementation reflects the requirement for secure, local-first data handling identified during system analysis (Chapter 3) and supports the analytical and adaptive features defined in the system design (Chapter 4).

Core Components

The database subsystem is implemented across three primary modules within the database/ package:

- **DbManager** (`"db_manager.py"`)
    - Central database access layer
    - Handles all read and write operations
    - Manages database lifecycle and connections
- **Schema Definitions** (`"schema.py"`)
    - Defines all database tables using SQLAlchemy ORM models
    - Provides a structured representation of system data
- **Encryption Support** (`"sqlcipher_support.py"`)
    - Handles optional SQLCipher encryption
    - Manages key generation and secure storage

Database Structure

The system uses a relational SQLite database, with tables designed to support both real-time monitoring and long-term behavioural analysis.

The primary tables include:

- **ActivityLog**
    - Stores one record per monitoring cycle (every 60 seconds)
    - Contains application data, idle time, and classification
- **DailyFocusBreakdown**
    - Maintains aggregated daily statistics for focused, neutral, and distracted activity
    - Tracks streaks and behavioural metrics

- **ADHDInsights**
    - Stores higher-level behavioural patterns such as hyperfocus sessions and task switching
    - Aggregated on a per-day basis
- **UserSettings**
    - Stores user preferences and ADHD-related thresholds
- **ReminderResponse**
    - Records user interaction with system reminders
    - Captures response behaviour and effectiveness
- **BanditEvent**
    - Stores adaptive system decisions and reward signals
    - Supports reinforcement learning for reminder optimisation

This schema allows the system to operate across multiple levels, from raw activity capture to high-level behavioural analysis.

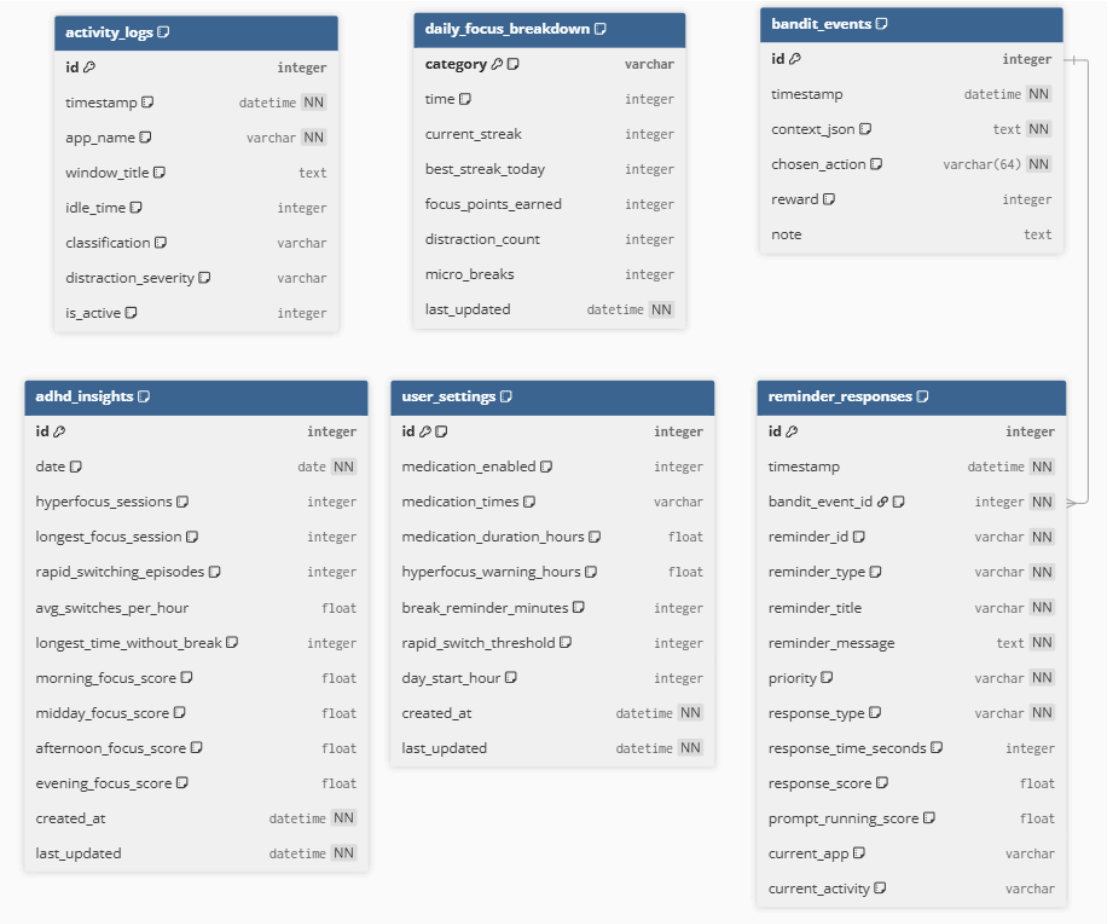


Figure 17: Database Schema

Data Flow Implementation

Write Path (During each monitoring cycle):

The write path illustrates how activity data is captured and persisted within the system, as shown in Figure 18.

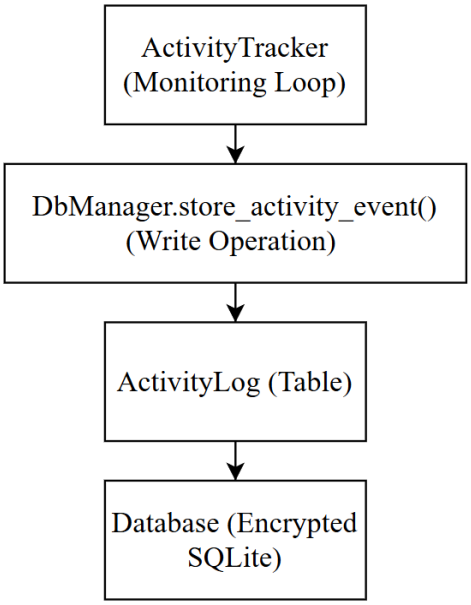


Figure 18: Data Write Path

Read Path (When data is requested by the user interface):

The read path illustrates how stored data is retrieved, processed, and returned to the user interface, as shown in Figure 19.

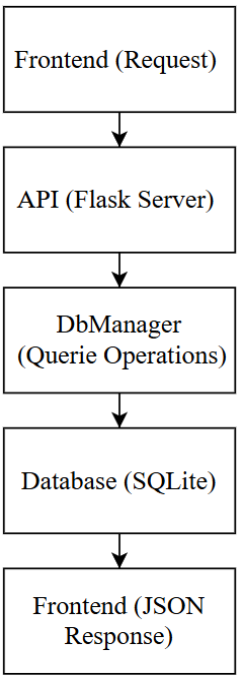


Figure 19: Data Read Path

Adaptive System Integration

In addition to standard read and write operations, the database also supports the adaptive reminder system.

Bandit System → BanditEvent (Table) → Reward Update → Database (SQLite)

- Each decision is stored along with its contextual data
- User responses generate reward signals based on subsequent behaviour
- These rewards are used to refine future system decision

Security and Local Data Handling

A key design requirement of the system is that all user data remains local to the device.

To support this:

- The database is stored locally on the user's machine
- SQLCipher encryption is used when available
- Encryption keys are generated securely and stored using OS-level protection mechanisms

This ensures that sensitive behavioural data is not exposed externally, aligning with privacy-focused design principles established earlier in the project.

Role Within the System

The database subsystem forms the backbone of the system's data pipeline:

Monitoring → Classification → Database → Analytics / UI / Adaptive System

By storing structured activity data, behavioural classifications, and adaptive system interactions, this component enables all higher-level functionality within the system, including dashboard visualisation, ADHD-specific insights, and adaptive feedback generation.

5.2.4 Adaptive Reminder and Bandit System

The adaptive reminder and bandit subsystem is responsible for delivering contextually relevant reminders and continuously improving their effectiveness over time. This implementation reflects findings from the literature review (Chapter 2), which emphasised the importance of timing, tone, and personalisation in ADHD support systems, and builds upon the behavioural data pipeline established in earlier components.

Core Components

The subsystem is implemented across both backend and frontend modules:

- **Bandit Algorithm** (`"bandit.py"`)
    - Implements a contextual Upper Confidence Bound (UCB) algorithm
    - Defines four reminder strategies (arms) and scoring logic
- **Bandit Service** (`"bandit_service.py"`)
    - Connects the algorithm to the live database
    - Retrieves historical performance data and selects actions
- **Reminder Manager** (`"useReminderManager.ts"`)
    - Controls scheduling and execution of reminders
    - Handles interaction with the backend API
- **Reminder Content Library** (`"adhdReminders.ts"`)
    - Defines all reminder types and behavioural states
    - Provides context-aware message selection
- **Notification Interface** (`"ReminderNotification.tsx"`)
    - Displays reminders to the user
    - Handles user interaction and response logging

Bandit Algorithm Implementation

The adaptive reminder system is driven by a contextual Multi-Armed Bandit algorithm using the Upper Confidence Bound (UCB) strategy. This approach balances exploration (trying different reminder strategies) with exploitation (selecting strategies that have previously performed well).

At each decision point, the algorithm evaluates all available actions using three components:

Estimated Reward (Exploitation)

- The average historical reward for each action

Exploration Term

- Encourages the system to try less frequently used actions

Context Adjustment

- Applies minor adjustments based on the current user state

The scoring function is defined as shown in Figure 20

$$score = 0.65 \cdot \hat{\mu} + 0.35 \cdot prior + c \cdot \sqrt{\frac{2\ln(N+2)}{n+1}} + context$$

Figure 20: Reminder score calculation

Where:

- $\hat{\mu}$ is the mean reward for the action
- Prior is a predefined baseline probability for each action
- N is the total number of interactions
- n is the number of times the action has been selected
- context represents the heuristic adjustment based on current behaviour

The core decision logic of this scoring process is implemented in "`choose_action_with_ucb()`" as shown in Figure 21.

```
scores: Dict[str, float] = {}
for arm in arms:
    stats = arm_stats.get(arm, {})

    # How often this arm was selected.
    count = float(stats.get("count", 0.0))

    # Learned average reward for this arm (if never used, default 0).
    empirical = float(stats.get("mean", 0.0)) if count > 0 else 0.0

    # Prior baseline to avoid extreme cold-start behavior.
    prior = DEFAULT_PRIORS.get(arm, 0.0)

    # Blend learned performance and prior.
    base = (0.65 * empirical) + (0.35 * prior)

    # UCB exploration bonus:
    exploration = exploration_c * sqrt(max(0.0, (2.0 * log(total_count + 2.0)) / (count + 1.0)))

    # Final arm score combines global learning + local context heuristics.
    scores[arm] = base + exploration + arm_context_adjustment(arm, context)

# Pick the arm with highest final score.
best_action = max(scores, key=lambda arm: scores[arm])
```

Figure 21: UCB-Based Action Selection Loop

This implementation computes a score for each action using reward, prior, exploration, and context. Less frequently selected actions receive higher exploration values.

The highest-scoring action is selected and returned to the frontend, where "`useReminderManager.ts`" applies the returned configuration to schedule and display the next reminder.

Adaptive Reminder Decision and Learning Process

The system uses a contextual bandit approach to determine when and how reminders should be delivered, while continuously adapting based on user behaviour.

At each decision point:

1. The frontend constructs a behavioural context (e.g. activity classification, focus duration, idle time, and task-switching behaviour)
2. This context is sent to the backend via the API
3. The bandit service evaluates all available actions using the UCB scoring function
4. The highest-scoring action is selected and returned

The available actions include:

- Soft nudges (low priority, longer intervals)
- Focus prompts (moderate intervention)
- Break prompts (high-priority intervention)
- Suppression (no reminder shown)

Once a reminder is delivered, the system evaluates user response and subsequent behaviour to generate a reward signal.

The adaptive process can be summarised as follows:

Context → Bandit Decision → Reminder → User Response → Reward → BanditEvent (Table)

- User responses (acknowledged, dismissed, ignored) provide immediate feedback
- Follow-up behaviour (e.g. improved focus after a reminder) contributes to outcome-based reward
- All interactions are stored and used to update future decision-making

The system incorporates contextual signals such as activity classification, focus duration, idle time, task-switching frequency, and time of day. These inputs allow the bandit algorithm to adapt its behaviour to the user's current state, ensuring that reminders remain relevant and appropriately timed.

Role Within the System

The adaptive reminder subsystem forms the decision-making and feedback layer of the system:

Database → Bandit System → Reminder → User Behaviour → Database

By using stored behavioural data to influence future decisions, this component enables the system to transition from static monitoring to adaptive, user-centred behavioural support.

5.2.5 User Interface and Desktop Integration

The user interface and desktop integration subsystem is responsible for presenting system data and delivering user interactions through a standalone desktop application. This implementation supports the requirement for an accessible, low-friction interface identified in Chapter 3, and reflects design considerations from Chapter 2 regarding clarity, cognitive load, and ADHD-friendly interaction.

Core Components

The interface is implemented using an Electron-based desktop application combined with a React and TypeScript frontend:

- **Electron Main Process** (`"main.js"`)
    - Manages application lifecycle and window creation
    - Spawns the Python backend process
    - Handles notification rendering and positioning
- **React Frontend** (`"App.tsx"`)
    - Provides tab-based navigation and layout
    - Displays monitoring data and insights
- **Preload Bridge** (`"preload.js"`)
    - Exposes a secure API between the frontend and Electron
    - Enables safe communication without direct Node access
- **Notification System** (`"notification.html"`)
    - Displays reminder overlays

    - Handles user interaction with notifications

Desktop Integration

The system is deployed as a standalone desktop application using Electron, allowing it to run locally without reliance on a browser.

Electron (Main Process) → Python Backend → API → React Frontend → Electron UI

- The Electron main process launches the backend as a child process
- The frontend communicates with the backend via HTTP API calls
- Notifications are rendered as a separate “always-on-top” window
- IPC (Inter-Process Communication) is used in the Electron environment to communicate between processes, particularly for notifications and user interaction events.

User Interface Design and Visualisation

The interface is organised into four primary views, each designed to present different aspects of user behaviour in a clear and accessible manner.

Overview Dashboard (“`MonitoringDashboard.tsx`”)

The overview dashboard (Figure 22) provides a high-level summary of user activity, including focus score, hourly behaviour, and most-used applications. A bar chart is used to represent hourly activity because it allows users to quickly identify patterns of focus and distraction throughout the day.

The layout prioritises immediate readability, with key metrics such as focus score and current activity presented prominently. This supports quick interpretation without requiring detailed analysis, which is particularly important for users who may struggle with cognitive overload.



Figure 22: Overview Dashboard

ADHD Metrics (“ADHDMetrics.tsx”)

The ADHD metrics view (Figure 23) focuses on behavioural patterns specific to ADHD, including hyperfocus duration, task-switching frequency, and time-of-day performance. Visualisations are used to highlight trends rather than raw values, allowing users to better understand how their focus changes over time. This view separates behavioural insights from general activity data, reducing visual clutter and allowing users to engage with more detailed analysis only when needed.

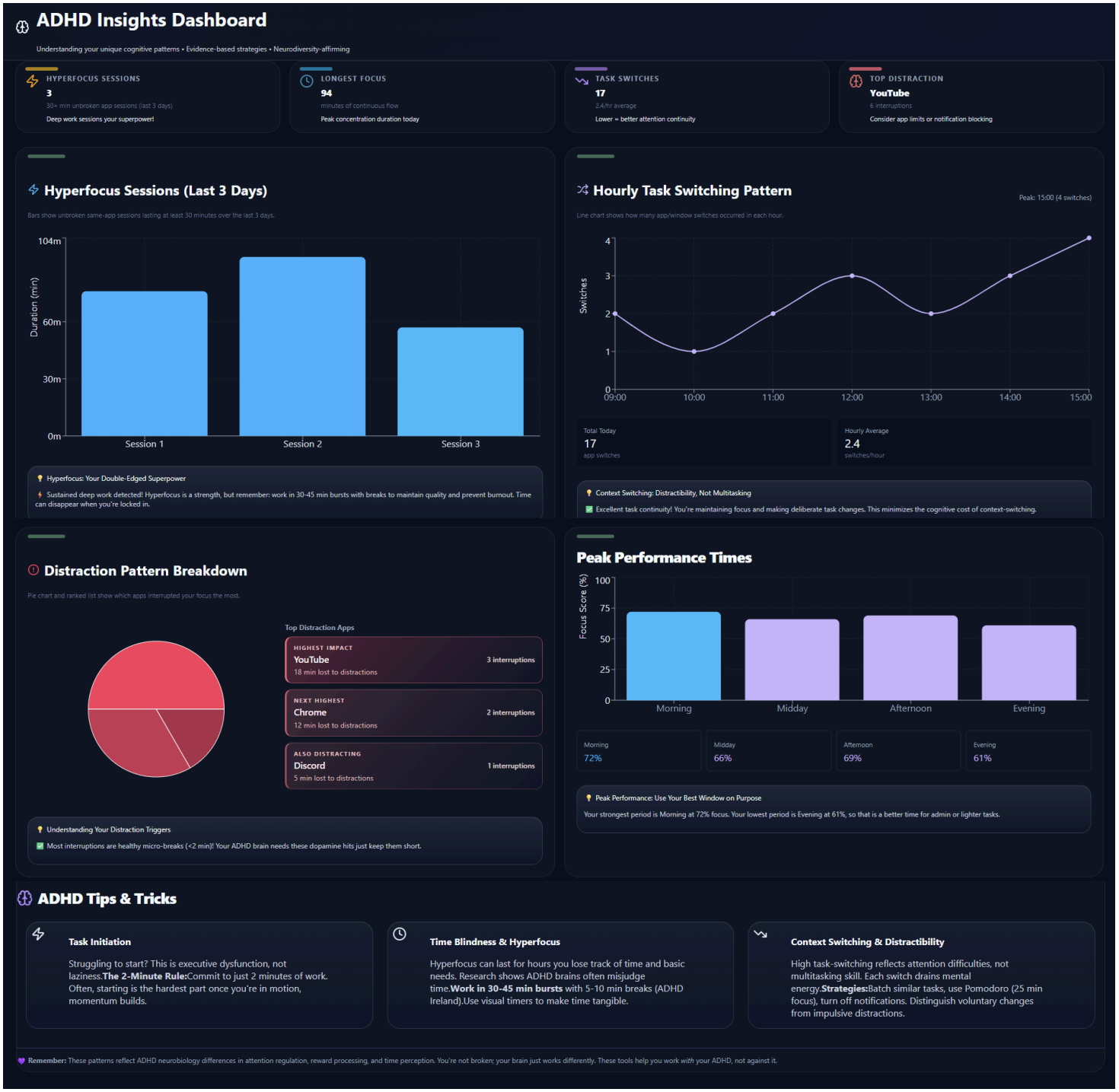


Figure 23: ADHD Metrics Dashboard

Reports Centre (“ReportsCenter.tsx”)
The reports interface (Figure 24) provides structured summaries of user activity over longer periods. This supports reflection and external use cases such as progress tracking or sharing results.

The design emphasises clarity and exportability, ensuring that generated reports are easy to interpret outside of the application environment.

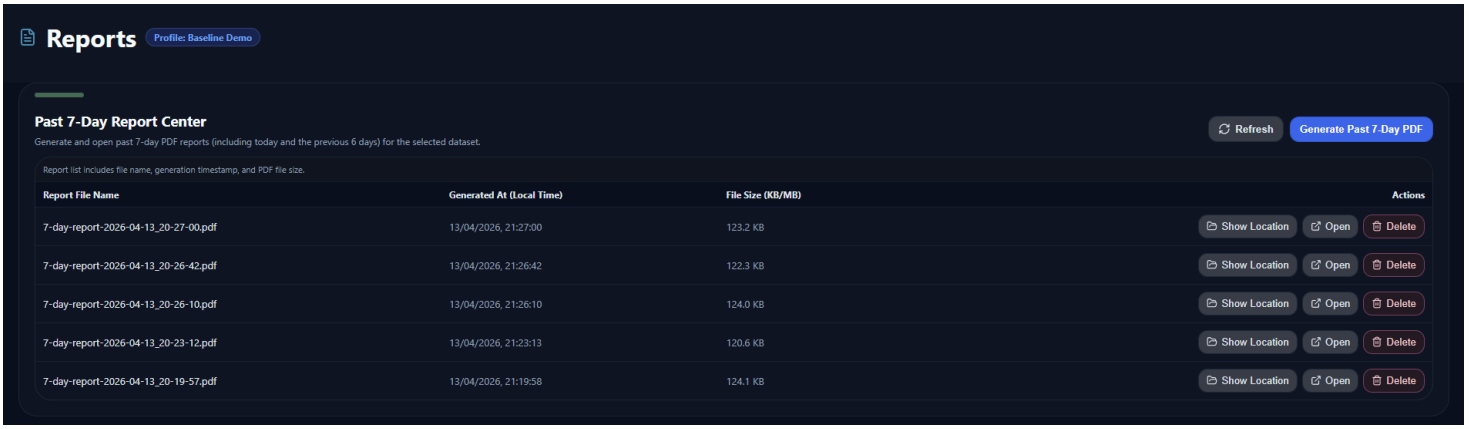


Figure 24: Reports Centre

Role Within the System

The user interface subsystem forms the interaction layer of the system:

User → Interface → API → System Components → Interface → User

By presenting behavioural data and delivering adaptive feedback, this component enables users to engage with and respond to the system, completing the feedback loop established by earlier subsystems.

You currently end that section straight after screenshots and role within the system. I’d insert a short subsection before Role Within the System:

5.3 Deployment

The final system is deployed as a standalone desktop application, packaged as an executable (.exe) for end users. This approach removes the need for manual installation of dependencies such as Python or Node.js, allowing the application to be launched in a similar manner to standard consumer software.

The deployment package encapsulates all core components of the system, including the Electron-based interface, the compiled React frontend, and the Python backend. When the application is launched, the backend service is automatically initialised and managed internally by the Electron process,

ensuring seamless communication between system components without requiring user configuration.

During development, deployment was managed through script-based orchestration to handle process control, testing, and debugging. In the final implementation, this complexity is abstracted away through packaging, resulting in a simplified and user-friendly deployment model.

5.3 Conclusions

Together, these components form a complete pipeline that captures user behaviour, interprets it in context, and delivers personalised, adaptive support. The system integrates monitoring, classification, data storage, and reinforcement learning to operate as a continuous feedback loop.

The resulting implementation reflects the intended design of a local-first, privacy-conscious, and ADHD-aware productivity tool.

6. Testing and Evaluation

6.1. Introduction

This chapter has two primary purposes: to verify that the implemented system behaves correctly, and to evaluate how effectively it satisfies the project requirements and objectives.

Testing primarily focuses on the system's backend components, including activity monitoring, classification, data storage, API integration, and adaptive reminder behaviour. These areas form the core functionality of the system and are therefore the primary focus of validation.

Frontend components were validated through manual inspection and end-to-end execution of the system. In addition to testing, the system was evaluated based on how effectively it met the defined project objectives, including usability, responsiveness, and the relevance of generated reminders.

System behaviour was also assessed through practical use and observation, enabling evaluation of real-world performance, user interaction flow, and the system's overall usefulness in supporting healthier digital habits.

This combined approach ensures that the system’s core pipeline is thoroughly tested while also providing a broader evaluation of its effectiveness, usability, and limitations.

6.2. System Testing

This section outlines the approach used to test the system, focusing on validating both individual components and the overall system pipeline. It describes the testing strategy and how different testing levels were applied to ensure correct system behaviour.

6.2.1 Testing Strategy

The testing strategy for FocusFinder was designed to validate both the correctness of individual components, as explained in earlier chapters, and the system's functionality as a complete pipeline. Testing was conducted across three levels: unit testing, integration testing, and end-to-end validation testing.

Unit testing was used to verify the functionality of isolated backend components. This included core modules and features such as activity tracking, idle detection, window tracking, classification, database persistence, and the adaptive decision-making algorithm. Each component was tested independently using controlled inputs to ensure predictable, repeatable

behaviour. Where necessary, external dependencies were mocked to allow logic to be tested in isolation.

Integration testing was used to validate interactions between components, particularly the flow of data between monitoring, classification, storage, and the reinforcement learning system. These tests helped ensure that activity data was correctly captured, processed, and passed into the decision-making pipeline, and that all resulting actions were recorded consistently.

End-to-end validation was used to confirm that the full system pipeline operates correctly in practice. This included executing the complete monitoring and feedback loop using the live API. Activity data was recorded, processed, and used to generate adaptive reminders. Repeated decision-and-feedback cycles were used to verify that the system adapts over time in response to user interactions.

All automated tests were implemented using Python's unit test framework. This provided a consistent structure for test execution and enabled key components to be validated under controlled conditions. Where automated testing was not feasible, particularly for user interface interactions, manual validation was used to confirm expected behaviour.

6.2.2 Automated Backend Unit Testing

Automated unit testing focused on validating the behaviour of individual backend modules in isolation, ensuring that each component performs correctly before integration into the wider system.

The following modules were covered:

- **Idle Detection**
    - Idle duration calculation
    - Threshold-based idle state detection
    - Reset of activity timestamps
- **Window Tracking**
    - Retrieval of active window information
    - Resolution of process names
    - Filtering of invalid or blank windows
    - Identification of the active window
- **Activity Tracking**
    - Recording of activity events with required fields
    - Retrieval of window metadata

    - Updating of monitoring intervals
- **Activity Classification**
    - Rule-based classification of applications
    - Keyword-based inference from window titles
    - Browser-specific heuristics
    - Handling of edge cases (e.g. unknown or missing inputs)
- **Adaptive Decision-Making (Bandit Algorithm)**
    - Correct action selection under different contexts
    - Avoidance of inappropriate suppression
    - Convergence towards higher-reward actions over repeated cycles
- **Database Persistence**
    - Writing and retrieving activity records
    - Preservation of multiple entries
    - Data consistency across sessions
- **Performance and Response Time**
    - Activity classification completes within a defined time threshold
    - Bandit action selection completes within a defined time threshold
    - Database write operations complete within a defined time threshold
- **Reminder Content Integrity**
    - All bandit action types reference only known, supportive reminder categories
    -

These tests ensure that each backend component behaves correctly in isolation before integration into the full system pipeline.

6.2.5 Test Coverage Of Functional Requirements

Req ID	Requirement	Validation Method	Result	Notes
FR 1	Start and stop activity monitoring	Manual validation through repeated runtime interaction with monitoring controls	Pass	Monitoring controls confirmed to correctly start and stop activity tracking; however, this behaviour is not covered by automated tests
FR 2	Track active applications, task switches, and idle periods	Automatic unit testing using `"test_idle_detector.py"`, `"test_window_tracker.py"`, and `"test_activity_tracker.py"`	Pass	Core monitoring behaviour validated with isolated and repeatable test inputs, including idle detection, window tracking, and activity logging
FR 3	Store recorded activity data in a local data store	Automatic persistence testing using `"test_db_manager.py"` with a real SQLite database, combined with activity tracker and pipeline tests	pass	Storage behaviour validated end-to-end: events are written and retrieved correctly, multiple inserts are preserved, and stored data is reflected accurately in summaries
FR 4	Classify recent activity as focused, neutral, or distracting	Automatic unit testing using `"test_activity_classifier.py"`, covering rule-based classification, keyword heuristics, browser logic, and edge cases	pass	Classification logic validated across all major code paths, including known applications, keyword inference, and edge cases such as sentinel values and empty inputs
FR 5	Infer potential distraction from behavioural signals	Automatic context-based testing using `"test_bandit_smoke.py"`, including signal threshold scenarios	pass	Behavioural signals validated through controlled scenarios: high task-switching triggers intervention, while calm, focused states allow suppression of reminders
FR 6	Select and deliver reminders based on context, timing rules, and past responses	`Automatic testing using "test_bandit_smoke.py"` and API validation loop with repeated reward cycles	Pass	Decision-making logic and adaptive behaviour were validated, demonstrating correct action selection and improvement over repeated interactions
FR 7	Log the user's response to each reminder	Automatic API testing using choose/reward/events endpoints and event inspection	Pass	Backend logging confirmed to correctly record reminder events and associated user responses within the system data store
FR 8	Update the internal model based on reminder outcomes	Automatic adaptation testing using "test_bandit_smoke.py" and repeated reward loop validation	Pass	Learning behaviour is demonstrated through repeated cycles, with the system adapting towards higher-reward actions over time
FR 9	Present dashboards and reports	Manual validation through interaction with the running user interface	pass	Dashboard outputs verified against underlying stored data; functionality confirmed, though no automated frontend testing was implemented

6.2.6 Test Coverage Of Non-Functional Requirements

NFR ID	Requirement	Validation Method	Result	Notes
NFR 1	All data is stored locally	Architectural verification and runtime inspection	Pass	System operates entirely on the local machine using SQLite; no external network calls are performed
NFR 2	Monitoring runs unobtrusively	Automated micro-performance testing using `"test_performance.py"`, combined with runtime observation	Pass	Core backend operations (classification, bandit decision, and database writes) complete well below perceptible thresholds (≤100ms for logic, ≤500ms for writes), supporting unobtrusive background operation; full-system resource profiling not performed
NFR 3	Minimal CPU and memory usage	Automated micro-performance testing using `"test_performance.py"`	Partial Pass	Key operations execute quickly in isolation, indicating lightweight backend logic. However, no full CPU/memory profiling was conducted under long-term monitoring conditions
NFR 4	Simple and easy-to-use UI	Manual walkthrough of dashboard and controls	Pass	Core functionality is accessible and understandable without training; no formal usability study was conducted
NFR 5	Adaptive behaviour over time	Automated testing using bandit adaptation tests and API loop	Pass	The system demonstrates learning behaviour, improving action selection based on reward feedback
NFR 6	Reliability and data integrity	Automated restart persistence test in `"test_db_manager.py"`, combined with repeated execution	Pass	Data persistence validated across simulated application restart by closing and reinitialising the database manager; no crash-recovery or stress testing performed
NFR 7	Platform compatibility (Windows)	Local deployment and execution testing	Pass	System runs correctly in the intended Windows environment; cross-platform support not required
NFR 8	Supportive, non-judgemental feedback tone	Automated content validation using `"test_reminder_tone.py"`, combined with manual review and extended real-world usage	Partial Pass	Reminder content automatically validated against a list of harsh or judgmental language, ensuring consistency. The system was also used over an extended period during development, with no negative or overly critical tone observed in practice. However, no extensive formal user-based evaluation was conducted

6.2.7 Summary of Testing

Testing across the entire system confirms that the core functional requirements have been successfully implemented and validated. Automated unit tests provide strong coverage of backend components, while integration and end-to-end validation confirm that the full system pipeline operates correctly in practice.

Non-functional testing further supports system performance, ensuring key operations execute within acceptable time thresholds and that data persistence is verified across sessions.

Some limitations remain, particularly in automated frontend testing and full system resource profiling. However, these do not reduce the effectiveness of the system's core functionality.

6.3 System Evaluation

This section evaluates the completed system against the defined project objectives, examining how effectively it meets its intended goals. It considers both the implemented functionality and the system's observed behaviour in practice, highlighting strengths, limitations, and overall effectiveness.

6.3.1 Evaluation Against Project Objectives

Improve user focus over time through adaptive support

Objective:

The system is designed to improve user focus by adapting its behaviour based on user interaction.

Implementation:

This is achieved through the reinforcement learning component, specifically the Multi-Armed Bandit model, which adjusts reminder selection based on user responses over time.

Validation :

The system's ability to provide real-time support is demonstrated through its continuous monitoring and immediate response to behavioural changes.

This is supported by testing of the reinforcement learning component (Section 6.2.5), where:

- Action selection is validated under different behavioural contexts
- Repeated reward cycles show convergence towards higher-reward actions

In addition, the system was used continuously throughout the development process, during which it was observed to adjust its behaviour in response to user interaction and to support sustained focus during development tasks. This provides qualitative support that the adaptive mechanism functions as intended in a real-world usage context.

Evaluation:

This confirms that the system successfully achieves its intended adaptive functionality. In individual use, the system was observed to support sustained focus during development, suggesting that the approach is effective in practice within at least a single-user context.

Limitations:

However, it is not possible to confirm that these results in improved focus for all users. Without large-scale, long-term evaluation across a diverse user group, the system's effectiveness in improving focus cannot be generalised. As a result, while the system demonstrates the correct adaptive behaviour, its long-term impact on user focus remains unproven beyond limited individual use.

Reduce reliance on external productivity tools

Objective:

The system aims to reduce reliance on external productivity tools by encouraging users to develop internal self-regulation skills.

Implementation:

This is supported through the system's continuous activity monitoring and behavioural reporting, which provide users with insight into their own habits. The dashboard and activity logs allow users to observe patterns in their focus, distraction, and task-switching behaviour over time.

Validation:

The system's ability to increase user awareness is demonstrated through its consistent recording and presentation of behavioural data over time.

This is supported by testing of the activity tracking and reporting components (Section 6.2.5), where:

- Activity tracking and logging are validated to ensure user behaviour is consistently recorded (FR2, FR3)
- Dashboard outputs are manually validated against stored data to ensure accurate and consistent representation of activity (FR9)

In addition, during development use, the system was observed to highlight patterns in focus, distraction, and task-switching behaviour, providing qualitative support that the presented data is meaningful and interpretable to the user.

Evaluation:

By making patterns such as frequent distractions, time-of-day effects, and task-switching behaviour visible, the system provides the user with the information needed to reflect on their habits. This supports the development of self-awareness, which is a necessary step towards self-regulation.

Limitations:

However, there is no direct evidence that users will reduce their reliance on external productivity tools as a result. Achieving this evidence would require long-term observation of user behaviour outside the system.

In addition, the effectiveness of this objective depends on the user actively applying insights gained from the system. While the system highlights patterns and areas for improvement, it does not enforce behavioural change and therefore, relies on the user to translate these insights into practical actions.

As such, while the system supports self-reflection, its impact on long-term independence from other tools cannot be confirmed within the scope of this project.

Provide real-time adaptive support for attention difficulties

Objective:

The system aims to provide real-time support for users with attention difficulties by delivering immediate, context-aware interventions during moments of distraction.

Implementation:

This is achieved through continuous activity monitoring and adaptive reminder delivery. The system tracks user behaviour in real time and triggers interventions based on detected behavioural signals, such as task switching, inactivity, or distraction patterns.

Validation:

The system's adaptive behaviour is demonstrated through changes in reminder selection over repeated interactions.

This is supported by testing of the reinforcement learning component (Section 6.2.5), where:

- Activity monitoring continuously tracks application usage and behavioural signals (FR2)
- Reminder delivery is triggered in response to detected distraction patterns (FR6)
- Performance testing confirms that classification and decision-making occur within sub-100ms thresholds (NFR2)

In addition, the system was used continuously throughout the development process, during which reminders were observed to trigger in direct response to distraction events. This provides qualitative support that the system operates in real time during practical use, although it does not constitute proof of effectiveness across different users.

Evaluation:

The system successfully provides real-time support, with interventions occurring during moments of distraction rather than retrospectively. In practice, reminders and feedback are delivered in context, allowing the system to actively support the user while tasks are being performed.

Limitations:

While the system operates in real time, its effectiveness in supporting users with ADHD has not been formally evaluated through extensive user studies. As a result, although real-time functionality is clearly demonstrated, its impact on users with varying levels of attention difficulty cannot be fully assessed within the scope of this project.

Increase user awareness of focus and distraction patterns

Objective:

The system aims to enable users to better understand their own behaviour by making patterns in focus, distraction, and task-switching visible over time.

Implementation:

This is achieved through continuous activity logging and the presentation of this data via dashboard visualisations. The system records user activity and aggregates it into summaries that highlight behavioural trends.

Validation:

The system's ability to increase user awareness is demonstrated by presenting behavioural patterns over time.

This is supported by testing of the data recording and reporting components (Section 6.2.5), where:

- Activity logging and storage are validated to ensure behavioural data is consistently recorded (FR3)
- Dashboard outputs are manually validated against stored data to ensure accurate representation of activity (FR9)

These results confirm that behavioural data is reliably captured and presented, allowing patterns in focus, distraction, and task-switching behaviour to be observed.

Evaluation:

The system successfully exposes behavioural patterns such as frequent task-switching, periods of distraction, and variations in focus over time. This provides the user with clear insight into their habits, supporting self-reflection and increased awareness.

Limitations:

While behavioural data is clearly presented, no formal usability study was conducted to evaluate how effectively users interpret or act on this information. As a result, the system's impact on improving user awareness cannot be fully assessed beyond observed functionality.

Deliver supportive, non-judgemental feedback

Objective:

The system aims to deliver feedback in a supportive, non-judgemental manner, ensuring that reminders motivate rather than discourage users, particularly those sensitive to negative or critical messaging.

Implementation:

This is achieved through predefined reminder content and controlled tone design. All notifications are structured to avoid negative or critical language, using neutral, encouraging phrasing across all reminder types.

Validation:

The system's ability to deliver supportive, non-judgemental feedback is demonstrated through the consistency of reminder tone across all interactions.

This is supported by testing of the reminder content validation component (Section 6.2.6), where:

- Reminder messages are checked against a predefined list of harsh or judgmental language to ensure none is present (NFR8)
- All reminder types are validated to ensure they use only approved, supportive phrasing

These results confirm that the system's feedback consistently adheres to the intended supportive tone.

In addition, during development use, no negative or critical phrasing was observed in reminders, providing qualitative support that the tone remains appropriate in practical usage.

Evaluation:

The system successfully maintains a consistent and supportive tone across all feedback. This aligns with the objective of encouraging user engagement without introducing discouraging or negative messaging.

Limitations:

While the tone is validated through automated checks and individual usage, not all users may perceive or respond to this feedback style in the same way. As a result, its effectiveness across a wider range of users cannot be fully assessed within the scope of this project.

Address limitations of static productivity tools

Objective:

The system aims to address the limitations of traditional productivity tools by replacing fixed, one-size-fits-all approaches with a system that adapts to individual user behaviour and changing needs.

Implementation:

This is achieved through the adaptive decision-making component, specifically the Multi-Armed Bandit model, which selects and adjusts reminder strategies based on user interaction rather than relying on predefined or static rules.

Validation:

The system's ability to move beyond static behaviour is demonstrated through its continuous adaptation of reminder strategies over time.

This is supported by testing of the adaptive decision-making component (Section 6.2.5 and 6.2.6), where:

- Adaptive reminder selection is validated under different behavioural contexts (FR6)
- Model updates are confirmed based on user response (FR8)
- Repeated interaction cycles demonstrate improved action selection over time (NFR5)

These results confirm that the system does not rely on fixed logic, but instead continuously adjusts its behaviour based on user interaction.

In addition, during development use, the system was observed to vary its reminder strategies over time, providing evidence that it adapts in practical use.

Evaluation:

The system successfully moves beyond static productivity models by introducing adaptive behaviour. Reminder strategies evolve over time in response to user interaction, allowing the system to better align with individual user patterns and preferences.

Limitations:

Although adaptive behaviour is clearly demonstrated, its effectiveness relative to traditional static productivity tools has not been formally evaluated. No comparative user study was conducted; therefore, it cannot be confirmed that the adaptive approach yields improved outcomes across users or use cases.

6.3.2 Key Strengths of the Completed Work

The completed system demonstrates several key strengths aligned with the project objectives

Modular System Architecture

A key strength of the system is its modular design, which separates monitoring, classification, storage, and adaptive decision-making components.

This is a strength because it allows each component to be developed and tested independently, improving maintainability and reducing the likelihood of system-wide failure. It also enabled targeted testing (Section 6.2), contributing to the reliability of the overall system.

Strong Backend Validation and Reliability

A major strength of the system is the depth of backend testing and validation.

This is a strength because it provides clear evidence that core functionality has been implemented correctly and behaves consistently under controlled conditions. The use of automated testing across multiple components ensures that errors can be identified and isolated, increasing confidence in the system's stability.

Adaptive Feedback Mechanism (Primary Strength)

The most significant strength of the system is its adaptive feedback mechanism.

This is a strength because it directly addresses the core limitation of traditional productivity tools by allowing the system to adjust its behaviour based on user interaction. Unlike static systems, the use of a Multi-Armed Bandit model enables continuous improvement in intervention selection, as validated through repeated interaction cycles (FR6, FR8, NFR5).

Real-Time Responsiveness
Another key strength is the system's ability to operate in real time.

This is a strength because it allows interventions to occur during moments of distraction, rather than after the fact. Performance testing (NFR2) confirms that the system responds without noticeable delay, making it suitable for continuous background use without disrupting the user.

Supportive and User-Centred Feedback Design
The system's feedback design is also a notable strength.

This is a strength because it ensures reminders are delivered in a supportive, non-judgmental manner, reducing the risk of discouraging users. Automated validation (NFR8) ensures consistency of tone, while practical usage confirms that feedback remains appropriate in context.

6.4 Evaluation of Project Management

This section reflects on the effectiveness of the project's planning and execution, evaluating how well the development process supported the system's successful delivery.

6.4.1 Evaluate Project Plan

The original project plan was structured around a component-based, iterative approach, which proved appropriate for managing system complexity. Breaking the system into discrete components allowed development to proceed in manageable stages and supported incremental progress.

The inclusion of research phases before and throughout implementation was also effective, as it reduced uncertainty and helped avoid inefficient development decisions.

However, while the plan allowed for flexibility and iteration, it did not fully anticipate the time required to implement and refine more complex components, particularly the adaptive feedback system. As a result, the development effort became concentrated on core backend functionality, with less time available for areas such as user settings and UX personalisation.

Overall, the project plan was well-structured and realistic in its approach to development, but slightly optimistic about the extent of testing and evaluation that could be completed.

6.4.2 Evaluate Project Execution

Project execution prioritised implementing and validating the core backend system, particularly the adaptive feedback mechanism.

This was a deliberate and appropriate decision, as the reinforcement learning component is the project's central contribution. Ensuring this system was fully implemented and tested provided a strong foundation for overall functionality.

As a result, significant progress was made in backend development, testing, and validation, with all core functional requirements successfully implemented and verified.

However, this prioritisation led to reduced coverage in other areas. Frontend testing, formal usability evaluation, and comprehensive non-functional performance analysis were not completed to the same depth.

Despite these trade-offs, the project remained aligned with its core objectives, and the most critical system components were successfully delivered and validated.

6.5 Conclusion

Testing and evaluation show that the core FocusFinder pipeline was successfully implemented and operates as intended. The strongest validation exists for backend monitoring, activity logging, data storage, API integration, and adaptive reminder selection.

The system's most significant strength is its adaptive behaviour, with testing confirming that reminder strategies update over time in response to user interaction. Real-time monitoring and the delivery of supportive feedback were also shown to function effectively in practice.

In individual use, the system was observed to support sustained focus and adapt to user behaviour, providing qualitative evidence that the approach is effective in a real-world context. However, due to the limited scope of testing, it is not possible to determine how effectively the system performs across a wider user base. A large-scale, long-term evaluation would be required to confirm its impact on different users and use cases.

Coverage is weaker in areas such as automated frontend testing, extended usability evaluation, and broader non-functional performance analysis. However, these limitations do not undermine the successful implementation of the system's core functionality.

Overall, FocusFinder can be considered a fully functional, successful adaptive productivity support system, while still leaving clear opportunities for further improvement and refinement.

7. Conclusions and Future Work

7.1. Introduction

This chapter presents the final conclusions drawn from the development and evaluation of the FocusFinder system, building on the findings from Chapters 5 and 6. It summarises the key outcomes of the project, including technical implementation, research integration, and observed effectiveness in practical use.

Based on these conclusions, areas for future work are then identified, outlining potential improvements and extensions to the system beyond the scope of this project.

7.2. Conclusions

Project Outcome

This project successfully achieved its primary aim of developing an adaptive productivity support system capable of monitoring user behaviour and delivering personalised feedback.

The implemented system demonstrates a fully functional reinforcement learning approach based on the Multi-Armed Bandit model. Analysis of recorded data and extended real-world use (90+ hours) shows that the system correctly performs both exploration and exploitation phases, adjusting its behaviour over time in response to user interaction.

Observed data confirms that the system evaluates the effectiveness of feedback strategies not only by immediate user response but also by sustained behavioural outcomes. In particular, reminders are weighted according to their ability to sustain focus over time, rather than on short-term engagement alone.

As a result, the system successfully adapts to the individual user, refining its prompt selection based on measured effectiveness. This demonstrates that the adaptive mechanism operates as intended, achieving the core objective of behaviour-responsive productivity support.

Technical Implementation

The system was successfully implemented as a complete and cohesive pipeline, integrating all core components from activity monitoring through to adaptive feedback delivery.

Each stage of the pipeline, including monitoring, classification, data storage, and adaptive decision-making, was developed and verified independently through targeted testing. In addition, integration testing confirmed that data flows correctly between components, with each stage producing the expected transformations and outputs.

The technologies selected for the project were appropriate for the system requirements, supporting efficient real-time processing and maintaining a clean, modular architecture.

The system operates continuously in real-world use, demonstrating reliable interaction between components and consistent end-to-end behaviour. This confirms that the overall architecture is sound, with no breakdown points observed during extended operation.

Research Integration

A key strength of the project was the continuous integration of research throughout the development process, rather than limiting research to the initial design phase.

Research was revisited at multiple stages of development, particularly when design decisions or implementation challenges arose. This iterative approach ensured that key decisions were informed by established findings in areas such as ADHD, behavioural response to feedback, and adaptive intervention strategies.

As a result, the system was not developed as a purely theoretical or technical solution, but as a research-informed system grounded in real behavioural and psychological principles. This ensured that the design of the feedback mechanisms, adaptation strategy, and user interaction model were aligned with evidence on how individuals respond to feedback and support systems.

This approach was instrumental in shaping the overall system, allowing it to move beyond simple activity tracking and towards behaviourally informed productivity support.

Practical Effectiveness

The system demonstrated effectiveness in individual use, supported by over 90 hours of recorded real-world usage during development.

Over this period, the system exhibited clear adaptive behaviour, progressively favouring reminder strategies that resulted in statistically more sustained focus. It was also observed to adapt to individual focus patterns over time, refining its behaviour based on measured outcomes rather than remaining static.

This was accompanied by a measurable improvement in recorded focus levels throughout extended use, indicating that the system can positively influence user behaviour in practice.

In addition, the system's visualisations provided meaningful insights into my personal user behaviour, highlighting patterns such as time-of-day performance and sources of distraction. This enabled informed adjustments to working habits, further supporting improved focus.

Limitations

The primary limitation of this project lies in the scope of its evaluation. The system was tested in a single-user context over a limited period, which limits the ability to generalise its effectiveness. While the system demonstrably adapts to user behaviour, it cannot be confirmed that it will perform consistently across different users, particularly given the variability in attention patterns and responses to feedback.

A further limitation is the use of a fixed set of predefined prompts. Although these prompts were informed by research and personal insight, they are not dynamically generated or expanded by the system. This may limit the system's ability to fully adapt to edge cases or users whose responses differ significantly from those anticipated during design.

Finally, the long-term effectiveness of the system remains unvalidated. While improvements in focus were observed during development, it is unclear whether these effects would be sustained over extended periods or how the system would perform as user behaviour evolves.

Final Statement

Overall, this project demonstrates that adaptive, behaviour-driven productivity support systems are both technically feasible and practically effective within an individual context. By combining continuous monitoring, behavioural analysis, and reinforcement learning, FocusFinder moves beyond common static productivity tools to provide a system that actively supports users' decision-making and focus.

While further validation is required to confirm long-term and multi-user effectiveness, the results of this project establish a strong foundation for developing more personalised and responsive productivity systems in the future.

7.3. Future Work

Future work for this project would focus on extending validation, improving adaptability, and refining user interaction:

- **Large-scale user evaluation**

  Conducting testing with a broader, more diverse user base over extended periods would help validate the system's effectiveness beyond a single-user context. This would also provide insight into how different users respond to adaptive feedback strategies.

- **Enhanced adaptive model**

  The current Multi-Armed Bandit approach could be extended by incorporating additional contextual inputs, such as time of day, task type, and historical behaviour patterns, allowing the system to make more informed and personalised decisions.

- **Dynamic prompt generation**

  Expanding from a fixed set of predefined prompts to a system capable of dynamically generating or evolving prompts would improve adaptability and enable it to handle edge cases and varied user responses more effectively.

- **Long-term behavioural analysis**

  Further investigation is required to determine whether improvements in focus are sustained over time, and how the system performs as user behaviour evolves or stabilises.

- **User interface refinement**

  While current visualisations provide useful insights, further refinement could improve clarity, usability, and engagement, enabling users to better interpret and act on the information provided.

References

Baroni, A. et al. (2018) ‘Differential impact of immediate vs. delayed feedback on performance in ADHD’, Journal of Attention Disorders, 22(4), pp. 351–362.

Babinski, D. E., Kujawa, A., Kessel, E. M., Arfer, K. B. and Klein, D. N. (2019) ‘Sensitivity to peer feedback in young adolescents with symptoms of ADHD’, Journal of Abnormal Child Psychology, 47(4), pp. 605–617.

Beaton, D. M., Sirois, F. and Milne, E. (2022) ‘Experiences of criticism in adults with ADHD: A qualitative study’, PLOS ONE, 17(2), p. e0263366.

Hosseinnia, M., Pirzadeh, A., Nazari, A. and Heidari, Z. (2025) ‘Applications for the management of Attention Deficit Hyperactivity Disorder: A systematic review’, Frontiers in Public Health, 13.

Jensen, L. X., Bearman, M. and Boud, D. (2025) ‘Characteristics of productive feedback encounters in online learning’, Teaching in Higher Education, 30(1), pp. 69–83.

Kallweit, C. et al. (2021) ‘Adult ADHD: Influence of physical activation, stimulation, and reward on cognitive performance’, Journal of Attention Disorders, 25(6), pp. 809–819.

Kumar, H. et al. (2024) ‘Using adaptive bandit experiments to increase and investigate engagement in mental health’, Proceedings of the AAAI Conference on Artificial Intelligence, 38(21), pp. 22906–22912.

Li, L., Chu, W., Langford, J. and Schapire, R. E. (2010) ‘A contextual-bandit approach to personalised news article recommendation’, in Proceedings of the 19th International Conference on World Wide Web, pp. 661–670.

Nordby, E. S. et al. (2022) ‘The effect of SMS reminders on adherence in a self-guided internet-delivered intervention for adults with ADHD’, Frontiers in Digital Health, 4.

Reiter, A. et al. (2024) ‘Neural correlates of altered reward processing in adults with ADHD’, Journal of Neuropsychology, 18.

Schultz, B. K. et al. (2022) ‘Emotional responsivity and feedback processing in ADHD’, Clinical Psychological Science, 10(4), pp. 712–728.

Shaw, P. et al. (2024) ‘A review of attention-deficit/hyperactivity disorder research: Cognitive, neurobiological, and clinical insights’, Nature Reviews Psychology, 3.

Song, H. et al. (2025) ‘Investigating the relationship between physical activity and tailored behaviour change messaging’, arXiv preprint, arXiv:2506.07275.

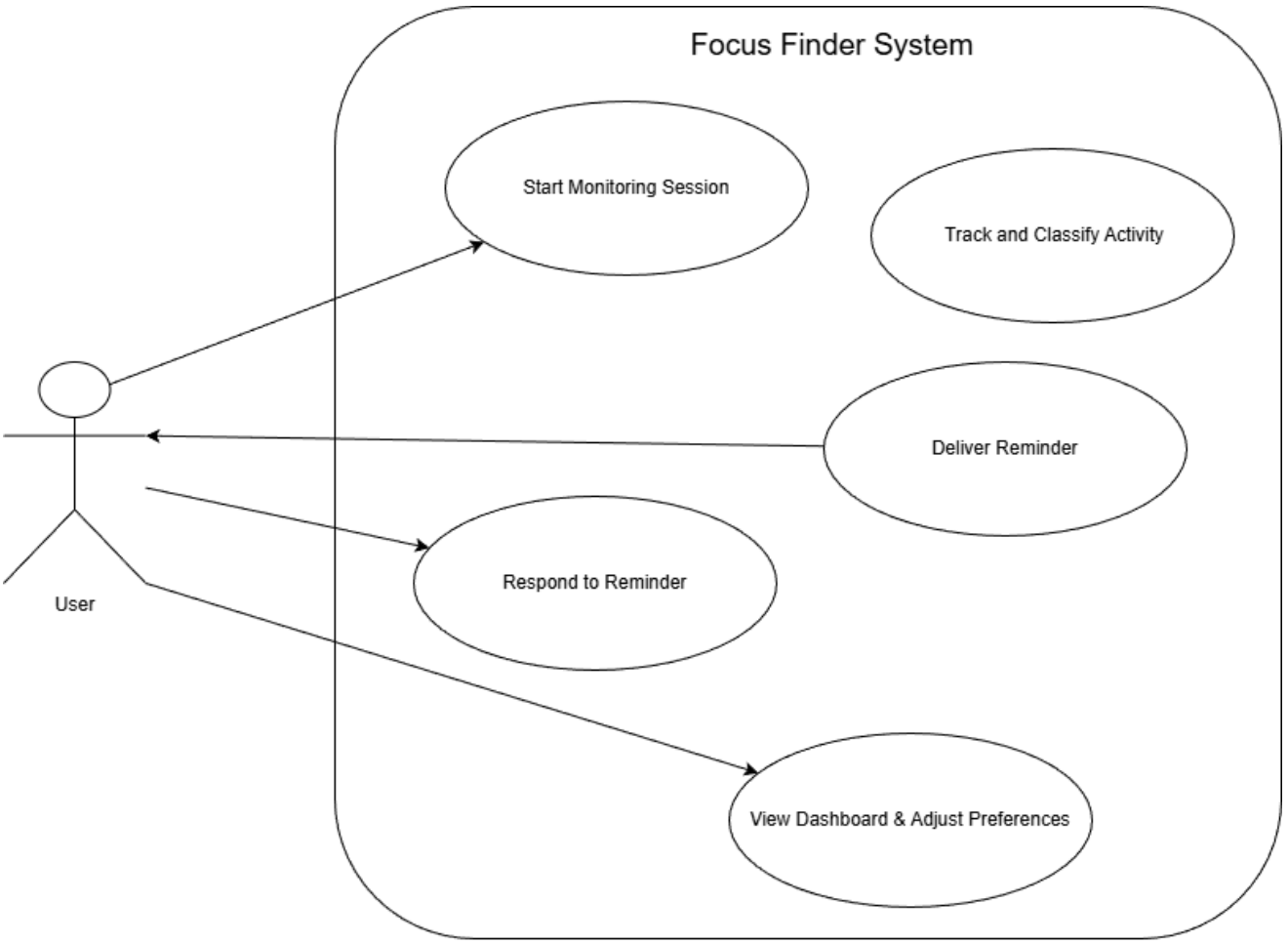
Staff, A. I. et al. (2023) ‘Teacher feedback, student ADHD behaviour, and the teacher–student relationship’, School Mental Health, 15(1), pp. 287–299.

Steur, A. J. and Seiter, M. (2021) ‘Properties of feedback mechanisms on digital platforms: An exploratory study’, Journal of Business Economics, 91(4), pp. 479–526.

Wirzberger, M. et al. (2024) ‘Optimal feedback improves behavioural focus during self-regulated computer-based work’, Scientific Reports, 14(1), p. 3124. doi: 10.1038/s41598-024-53388-3.

Appendix A: System Model and Analysis

A.1 Use Case Diagram



A.2 Use Case Narratives

USE CASE	1	**Start Monitoring Session**
Description of Goal in Context:	● This use case is triggered when the user wants Focus Finder to begin tracking their activity. ● It allows the system to collect OS activity signals in the background. ● It enables the adaptive reminder loop to begin operating.	
Preconditions:	● The application is installed and running. ● The user has opened the dashboard.	
Post Conditions, Success End Condition	1. Monitoring is active. 2. Activity logs are being recorded. 3. The system is ready to classify behaviour and deliver reminders.	
DESCRIPTION	This use case describes the user-initiated process that starts the monitoring component. Once the user enables monitoring in the interface, the background service begins collecting data on active windows, idle time, and app usage. The system transitions into its monitoring loop, preparing to classify behaviour and send adaptive reminders when necessary.	
Main Flow		
Step	**Action**	
1	User opens the Focus Finder dashboard.	
2	User selects “Start Monitoring.”	
3	System activates the monitoring component	
4	OS activity signals begin to be recorded.	
5	System transitions to Monitoring Active state.	
6	Dashboard updates to show tracking is running.	
ERROR Flow		
Step	**Branching Action**	
1.1.1	Monitoring service fails to start - System displays an error message.	
1.1.2	Local data store unavailable - System warns user and retries.	
Alternative Flow		
Step	**Branching Action**	
1.2.1	Auto-start monitoring is enabled - Monitoring begins automatically when the app opens.	

USE CASE	2	Track and Classify Activity
Description of Goal in Context:	● This use case occurs while monitoring is active. ● It allows the system to continuously observe user activity in the background. ● It enables behavioural classification to support adaptive decision-making.	
Preconditions:	● Monitoring session is active. ● System has access to OS activity signals.	
Post Conditions, Success End Condition	1. Activity data is recorded. 2. Behaviour is classified (focused, neutral, distracting). 3. Data is stored for future analysis and decision-making.	
DESCRIPTION	This use case describes the system's continuous background operation during an active monitoring session. The system collects activity data such as application usage, window focus changes, and idle time. This data is processed by the activity interpretation component, which classifies behaviour into meaningful categories. These classifications are stored and used to inform future adaptive feedback decisions.	
Main Flow		
Step	**Action**	
1	Monitoring session is active	
2	System observes OS activity signals	
3	Activity data is recorded	
4	System classifies behaviour	
5	Classified data is stored	
ERROR Flow		
Step	**Branching Action**	
2.1.1	Activity data cannot be retrieved - System skips current cycle and continues monitoring	
2.1.2	Invalid or missing data - System assigns default classification	
Alternative Flow		
Step	**Branching Action**	
2.2.1	No activity detected - System records idle state	

USE CASE	3	Deliver Reminder
Description of Goal in Context:	● This use case is triggered when the system detects distraction or inactivity. ● It allows the system to provide real-time feedback to the user. ● It enables adaptive intervention based on user behaviour.	
Preconditions:	● Monitoring session is active. ● Behaviour has been classified. ● Timing conditions for reminders are met.	
Post Conditions, Success End Condition	1. Reminder is delivered to the user. 2. User interaction is recorded. 3. System updates internal state for future decisions.	
DESCRIPTION	This use case describes how the system delivers adaptive reminders based on user behaviour. When distraction or inactivity is detected, the system evaluates the current context and selects an appropriate reminder using its adaptive decision-making component. The reminder is then presented to the user through a notification. The user's response is recorded and used to influence future system behaviour.	

Main Flow	
Step	**Action**
1	System detects distraction or inactivity
2	System evaluates behavioural context
3	Adaptive component selects reminder type
4	Reminder is delivered to the user
5	System waits for user response
ERROR Flow	
Step	**Branching Action**
3.1.1	No valid reminder available - System suppresses reminder
3.1.2	Notification fails to display - System logs failure and retries
Alternative Flow	
Step	**Branching Action**
3.2.1	User is focused - System suppresses reminder

USE CASE	4	**Respond to Reminder**
Description of Goal in Context:	● This use case occurs when a reminder is delivered to the user. ● It allows the user to respond to system feedback. ● It enables the system to learn from user interaction.	
Preconditions:	● A reminder has been delivered. ● The system is waiting for user input.	
Post Conditions, Success End Condition	1. User response is recorded. 2. System updates its internal model. 3. Future reminder behaviour is influenced.	
DESCRIPTION	This use case describes how the user interacts with system reminders. When a notification is presented, the user may acknowledge, dismiss, or snooze it. The system records this interaction and uses it as feedback for its adaptive decision-making component. Over time, these responses influence how and when future reminders are delivered.	

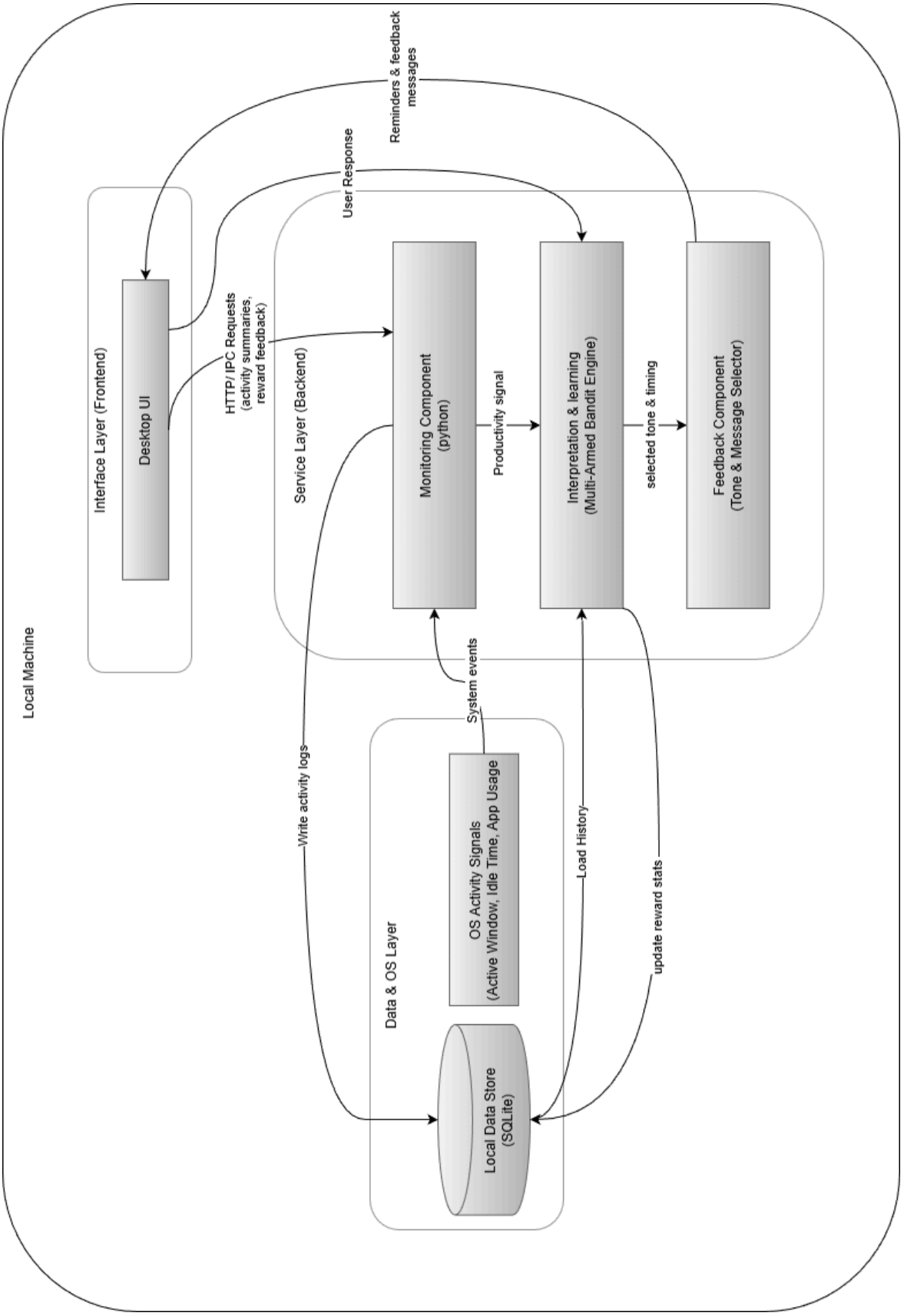
Main Flow	
Step	**Action**
1	Reminder is displayed to the user
2	User selects a response
3	System records the response
4	System updates internal state
5	System returns to monitoring state
ERROR Flow	
Step	**Branching Action**
4.1.1	Response cannot be recorded - System logs error and continues operation
3.1.2	Notification fails to display - System logs failure and retries
Alternative Flow	
Step	**Branching Action**
4.2.1	No user response - System treats reminder as ignored

USE CASE	5	View Dashboard and Adjust Preferences
Description of Goal in Context:	● This use case is triggered when the user wants to review recorded activity data. ● It allows the user to view summaries of focus, distraction, and task-switching behaviour. ● It also allows the user to adjust system settings such as reminder style and notification frequency.	
Preconditions:	● The application is installed and running. ● Activity data has been recorded.	
Post Conditions, Success End Condition	1. Dashboard data is displayed to the user. 2. User preferences may be updated. 3. Updated settings are stored and applied to future system behaviour.	
DESCRIPTION	This use case describes how the user interacts with the dashboard and system settings. The dashboard provides visual summaries of recorded activity, including focus trends, distraction patterns, and reminder history. The user may also adjust settings such as reminder style, tone, or notification frequency. Any updated preferences are stored by the system and used to influence future behaviour.	

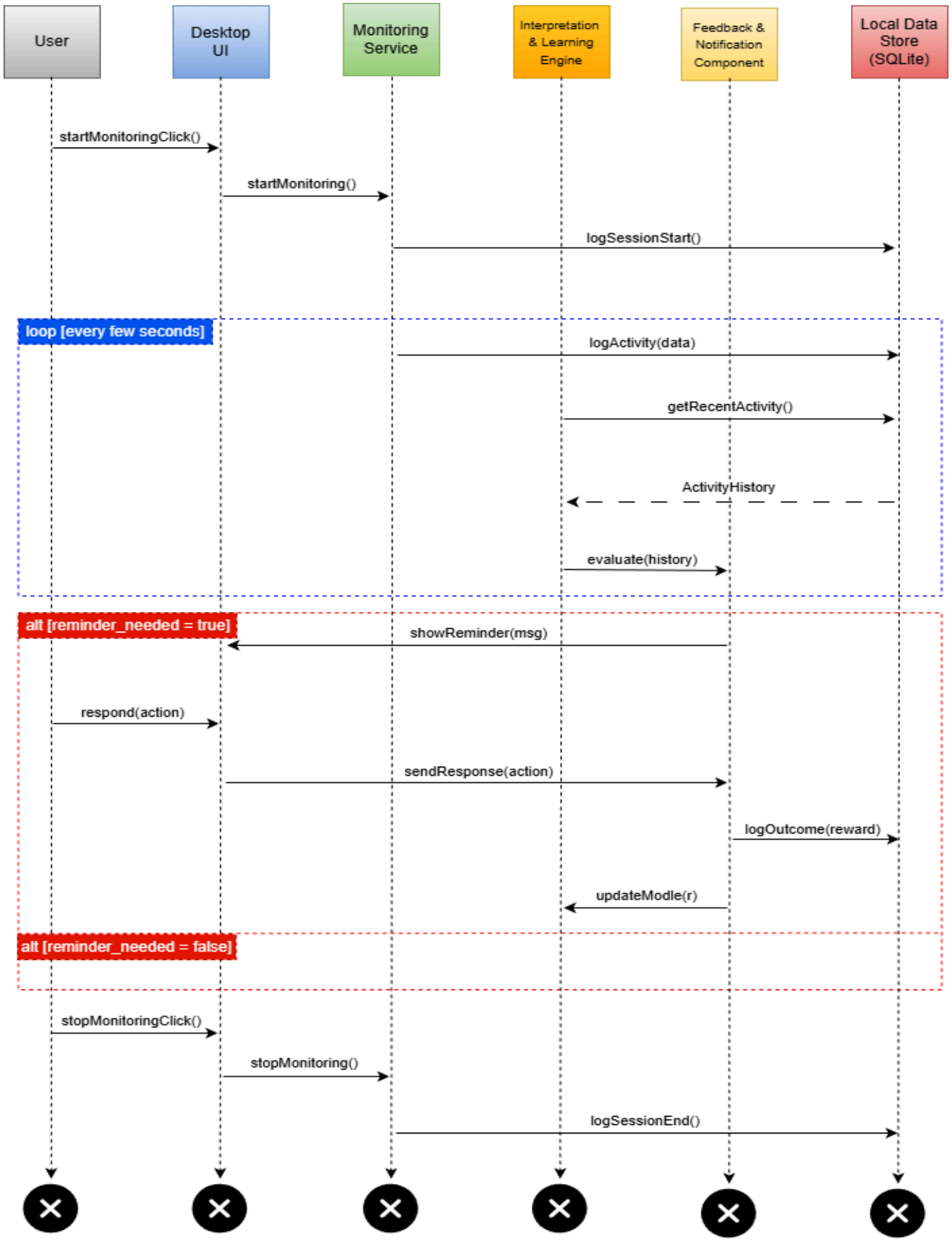
Main Flow	
Step	**Action**
1	User opens the dashboard
2	System retrieves stored activity data
3	Dashboard summaries are displayed
4	User reviews activity information
5	User adjusts one or more preferences
6	System saves updated settings
ERROR Flow	
Step	**Branching Action**
5.1.1	Activity data cannot be retrieved - System displays an error message
5.1.2	Settings cannot be saved - System warns the user and preserves previous settings
Alternative Flow	
Step	**Branching Action**
5.2.1	User views dashboard only - No settings are changed

Appendix B: Design

B.1 Full-size Technical Architecture Diagram



B.2 Sequence Diagram



Appendix C: Prompts Used with Gen AI

C.1 Accessibility and Writing Process Declaration

My written work is heavily reliant on speech-to-text software due to dyslexia. This allows me to express ideas verbally and capture my intended meaning in written form. Without this tool, it would be incredibly difficult for me to produce text that is clear, well-structured, and comprehensible.

In addition, Grammarly was provided to me by the TU Dublin Disability Support Service as an approved accessibility support. It was used throughout the writing of this report to assist with grammar, clarity, tone, and sentence structure. This tool does not generate original ideas or content; instead, it refines and improves the text I have produced as I write.

Both speech-to-text and Grammarly were used throughout this report, including during drafting and refinement. Their use is essential to ensure that my work is readable and accurately represents my own ideas.

It is important to distinguish these tools from generative AI systems. Generative AI tools (such as ChatGPT or Copilot) were not used to produce the written content of this report. Their use was limited to support tasks such as planning, structuring, and guidance, and all such usage is fully documented in this appendix.

Due to the consistent use of speech-to-text and Grammarly, the final written work may resemble AI-generated text when assessed by automated detection systems, as Grammarly determines “good structure” and wording. However, all ideas, arguments, and written content originate from my own work.

The use of these tools as accessibility support has been discussed with and approved by Ciaran O’Driscoll.

C.2 Use of AI (ChatGPT) in Report Development

Due to the iterative nature and duration of the project, the following prompts represent an inclusive, but not exhaustive, selection of those used during the development of this report.

C.2.2 Prompts Related to Clarity and Understanding

- “Does this section read correctly?”
- “Does this describe the system clearly?”
- “Am I explaining this concept properly?”
- “How can I improve clarity without changing meaning?”
- “Is this written in an academic tone?”
- “Is this explanation too vague or too detailed?”
- “Can this sentence be made clearer without changing its intent?”
- “Does this paragraph communicate the key idea effectively?”
- “Is the terminology used here appropriate for a technical report?”

C.2.3 Prompts Related to Structure and Organisation

- “How should I approach writing this chapter?”
- “What should go into this section?”
- “How do I structure this part of the report?”
- “Does this flow logically between sections?”
- “What is the best order to present these ideas?”
- “How can I improve the transition between these paragraphs?”
- “Does this section align with standard report structure?”
- “Am I repeating information unnecessarily here?”
- “Should this content be split into multiple subsections?”

C.2.4 Prompts Related to Reassurance and Validation

- “Is this good enough for a final year project?”
- “Does this meet expected academic standards?”
- “Am I on the right track with this section?”

- “Is this explanation too vague or acceptable?”
- “Does this demonstrate sufficient understanding of the topic?”
- “Is this level of detail appropriate for this section?”
- “Would this be considered a strong explanation in an academic context?”
- “Have I justified this decision clearly enough?”
- “Does this reflect a clear and logical argument?”

C.3 Use of AI (ChatGPT/ CoPilot) in Project Design (Prompt Examples)

The following prompts relate to the use of AI during the development of the system. In accordance with assessment guidelines, AI was used only for brainstorming, guidance, and conceptual clarification. AI was not used to generate system design, requirements, or analysis content.

C.3.1 Research Direction and Technology Selection

- “What are people usually using to track user activity on a desktop?”
- “Is there a straightforward way to monitor app usage locally?”
- “What kind of tools already exist for tracking productivity?”
- “What would I typically use in Python to handle something like this?”
- “Are there any limitations I should be aware of when tracking browser activity?”

C.3.2 System Analysis (Brainstorming Only)

- “What kind of questions should I be asking when gathering requirements?”
- “What do you think users actually care about in a system like this?”
- “What should I be thinking about when analysing user behaviour?”
- “How do I approach starting to figure out system constraints?”
- “What are some common non-functional requirements for something like this?”

C.3.3 Design Considerations (Conceptual Guidance Only)

- “What should I keep in mind when designing a feedback system?”
- “How do systems usually handle notifications without being annoying?”
- “What does modular design actually mean in practice?”
- “Any suggestions on how I can keep a system like this simple without losing functionality?”

C.4 Use of AI (CoPilot) in Coding

The following prompts are examples of how AI was used during implementation. GitHub Copilot was used as an iterative support tool throughout the coding process, assisting with debugging, clarifying syntax, implementing smaller code segments, and refactoring.

The development process followed a continuous refinement cycle in which implementation ideas were proposed, evaluated, and incrementally improved. AI-generated suggestions were reviewed, adapted, and validated before inclusion. All system design, architectural decisions, and final code behaviour were defined by me.

Due to the frequency and volume of interactions, it is not possible to provide an exhaustive list. The prompts below reflect the typical nature of this usage.

C.4.1 Debugging and Error Handling

- “Why is this function returning None when it should return a value?”
- “What does this Flask traceback mean and where is it failing?”
- “Why is this API route returning a 500 error?”
- “Why is this asynchronous call not resolving correctly?”
- “Why does this exception only occur when the database is queried the second time?”
- “Can you help fix why this function is not returning the correct value?”
- “Can you identify why this API call is failing?”
- “Can you help correct this SQL query so it works properly?”

C.4.2 Implementation and Syntax Support

- “What is the correct SQLAlchemy syntax for this type of query?”
- “How do I schedule a background task to run on a timed interval in Python?”
- “How do I correctly parse this JSON response from the backend?”
- “How do I pass a value safely from the Electron main process to the renderer?”
- “What is the correct way to expose this function through the preload script?”
- “How do I register this Flask route so that it accepts both GET and POST?”
- “What would a simple version of this function look like?”
- “Can you show an example of how to structure this API call?”
- “Can you help write a basic version of this function?”

C.4.3 Refactoring and Readability

- “Can this component be reorganised so the file structure is clearer?”
- “Is there a better way to structure these files so the project is easier to navigate?”
- “How can I separate this logic into cleaner modules?”
- “Can this component be refactored so responsibilities are more clearly defined?”
- “How can I reduce coupling between these parts of the system?”

C.4.2 Refactoring and Readability

- “What’s a good way to display this data clearly in the UI?”
- “Why is the data in this component not updating properly?”
- “What’s a clean way to render this chart using Recharts?”
- “How should I handle loading and empty states for this view?”
- “How can I improve how this data is presented without changing its meaning?”
- “What’s a good way to organise multiple graphs on the same screen?”